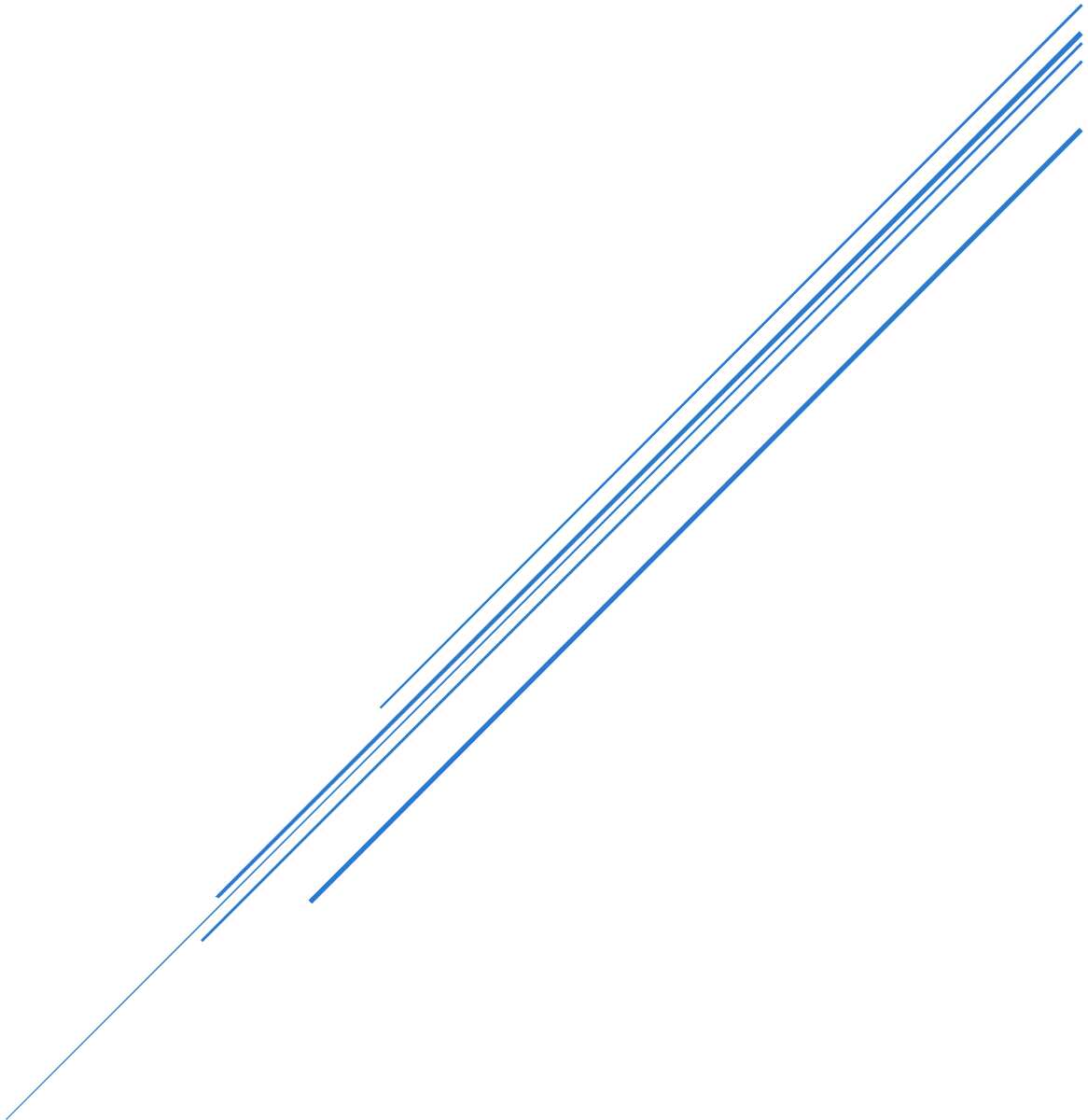


CSYM026 – GROUP TECHNICAL REPORT



Paddy Costello

Shah Abdul Qadir

Parastoo Shojaei Sarcheshmeh

Contents

GitHub Repository & Jira Board:	3
UML Class Diagrams – Author: Shah Abdul Qadir	3
Scoring User Stories & tasks – Author: Paddy Costelloe	7
Agile Scrum Report	8
Weekly Sprint – Author: Paddy Costelloe	8
Slack usage – Author: Paddy Costelloe	8
Agreed Workflow – Author: Paddy Costelloe	10
Client Review Process – Author: Paddy Costelloe	11
Managing GitHub and Jira – Author: Paddy Costelloe	11
Sprints and retrospectives overview	12
Code Breakdown	16
Ants and Bees classes – Author: Parastoo Shojaei Sarcheshmeh	16
Overview – Author: Parastoo Shojaei Sarcheshmeh	16
The Insect Base Class – Author: Parastoo Shojaei Sarcheshmeh	16
The Ant Abstract Class – author: Parastoo Shojaei Sarcheshmeh	16
Ant Subclasses – Author: Parastoo Shojaei Sarcheshmeh	17
The Bee Class – Author: Parastoo Shojaei Sarcheshmeh	18
The Place Class – Author: Parastoo Shojaei Sarcheshmeh	19
The AntColony Class – Author: Parastoo Shojaei Sarcheshmeh	20
The Hive Class – Author: Parastoo Shojaei Sarcheshmeh	20
Design Pattern: Observer Pattern – Mouse Event Systems	20
Singleton – QueenAnt – Author: Paddy Costelloe	20
Observer pattern – Author: Parastoo Shojaei Sarcheshmeh	21
Refactoring – Author: Shah Abdul Qadir	24
Migrating from Swing to JavaFX – Author: Shah Abdul Qadir	24
Thread Safety and Concurrency – Author: Shah Abdul Qadir	25
Dynamic UI Scaling – Author: Shah Abdul Qadir	25
Graphics Integration – Author: Shah Abdul Qadir	26
Audio System Refactoring – Author: Shah Abdul Qadir	27
Refactoring for Packaging – Author: Shah Abdul Qadir	27
Game System	29
Game Menus and Pane handling - Author: Parastoo Shojaei Sarcheshmeh	29
GameAppController - Author: Parastoo Shojaei Sarcheshmeh	29
MainMenuPane - Author: Parastoo Shojaei Sarcheshmeh	29
User Account System - Author: Parastoo Shojaei Sarcheshmeh	30
SeedGenerator - Author: Parastoo Shojaei Sarcheshmeh	30
HelpScreenPane - Author: Parastoo Shojaei Sarcheshmeh	30
GameScreenPane - Author: Parastoo Shojaei Sarcheshmeh	31
PauseMenuPane - Author: Parastoo Shojaei Sarcheshmeh	31
SettingsPane - Author: Parastoo Shojaei Sarcheshmeh	31
GameOverPane - Author: Parastoo Shojaei Sarcheshmeh	32
ScoreScreenPane - Author: Parastoo Shojaei Sarcheshmeh	32
Score and Timer Systems - Author: Parastoo Shojaei Sarcheshmeh	32

Graphics – Author: Shah Abdul Qadir	33
Audio – Author: Shah Abdul Qadir	33
Difficulty System – Author: Paddy Costelloe	33
Game Flow – Author: Paddy Costelloe.....	34
Testing.....	35
White Box Testing – junit testing - Author: Parastoo Shojaei Sarcheshmeh	35
Methodology - Author: Parastoo Shojaei Sarcheshmeh	35
Test Classes - Author: Parastoo Shojaei Sarcheshmeh	35
Black Box Testing – document – Author: Paddy Costelloe	38
Bugs & weaknesses	41
AI Usage Declaration	41
References	42
Appendix	43

GitHub Repository & Jira Board:

Suraj Ajit had access to these two sites. If further access is required for moderation contact
paddy.costelloe2@northampton.ac.uk

<https://github.com/thedirge0matic/Group1-CSYM026-AntsVsBees>

<https://software-engineering-csym026-group1.atlassian.net/jira/software/projects/SCRUM/summary>

The main branch ReadMe.md contains instructions for project set up and has a exe release build to download for windows.

UML Class Diagrams – Author: Shah Abdul Qadir

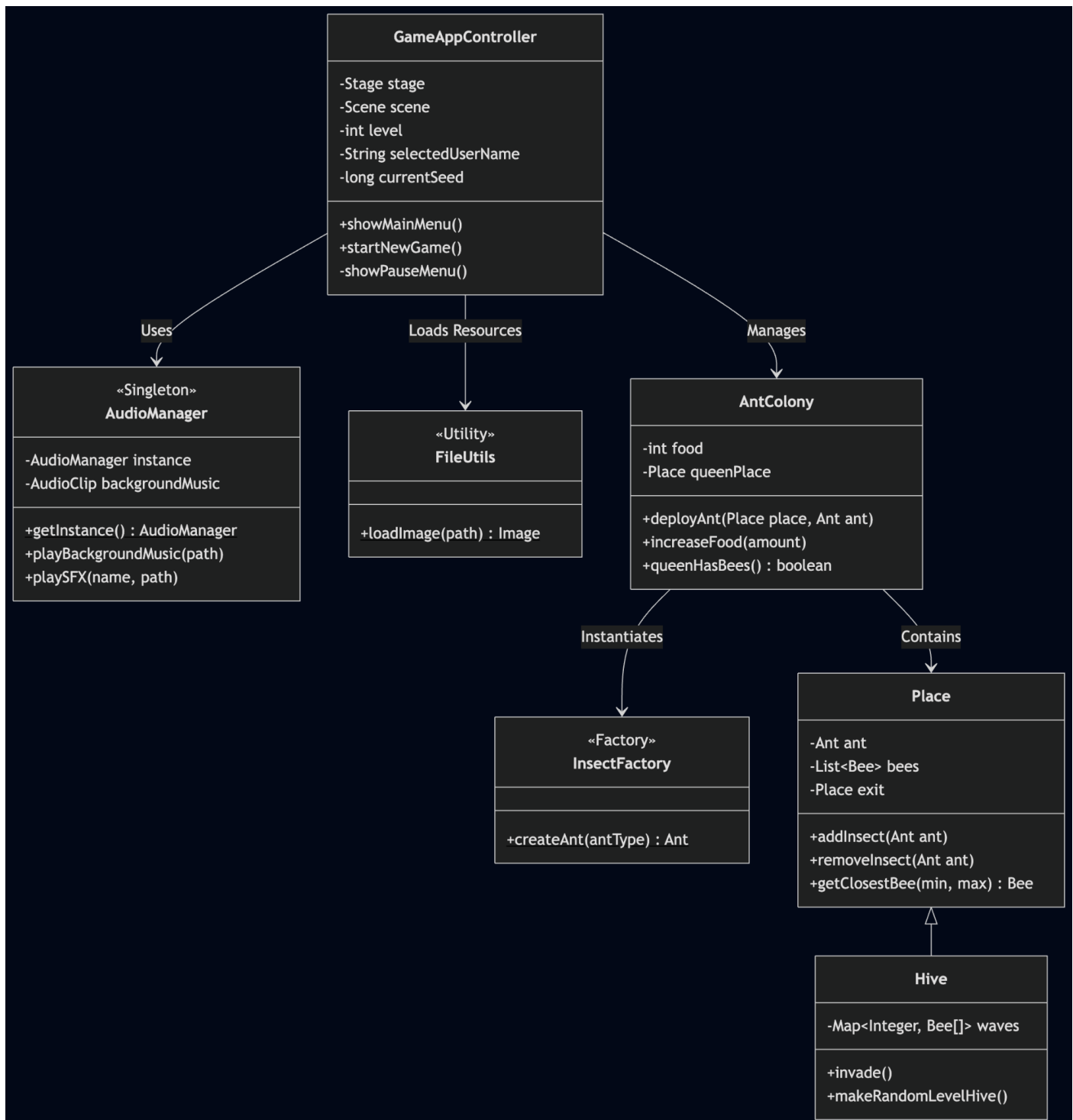
I put together the UML Class Diagram to map out our code structure. The game leans heavily on an inheritance tree for the bugs (Insect, Ant, Bee), but my main goal for the architecture was keeping the UI and file loading totally separate from the game rules.

To make sure the diagram matched our final code perfectly, I used my IDE to auto-generate a baseline map of all our classes. Then, I went in and organized it by hand, because auto-generated diagrams can get incredibly messy with big inheritance trees. I made sure to visually split the static utility classes (like FileUtils and ScoreManager) from the UI controllers like GameController.

I also highlighted specific design patterns. For example, I mapped out the Singleton pattern I used for the AudioManager. That pattern ensures only one instance of the audio engine exists at any time across the entire game, which is how I permanently fixed the music overlapping issue.

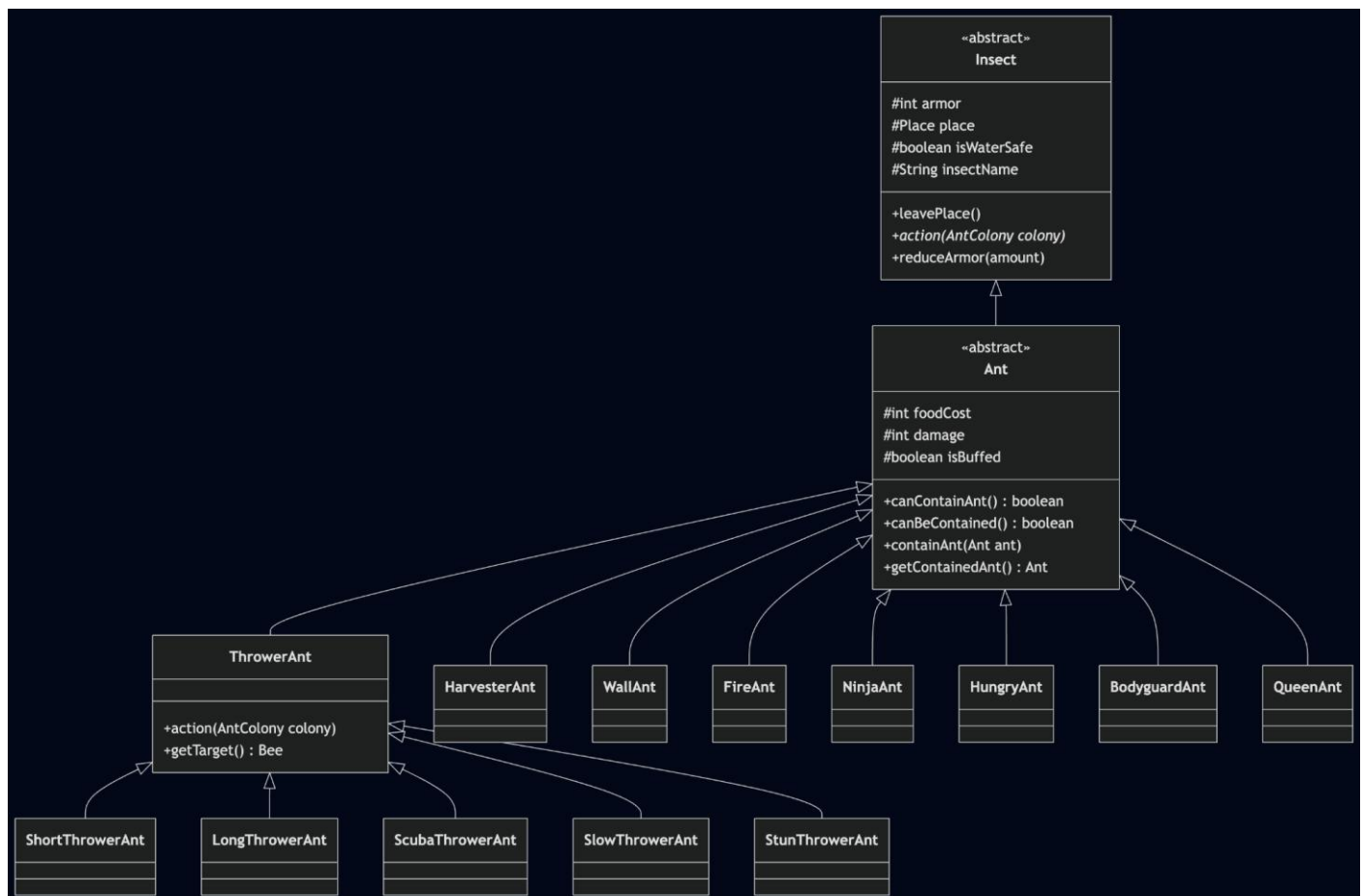
1. System Architecture and Managers

This first diagram shows how the different parts of the system are separated out. The `GameAppController` sits in the middle connecting the UI to the core game logic, and you can also see where I applied specific design patterns. The `AudioManager` runs as a Singleton and `FileUtils` is kept as a pure utility class, both of them completely separate from the actual `AntColony` game board.



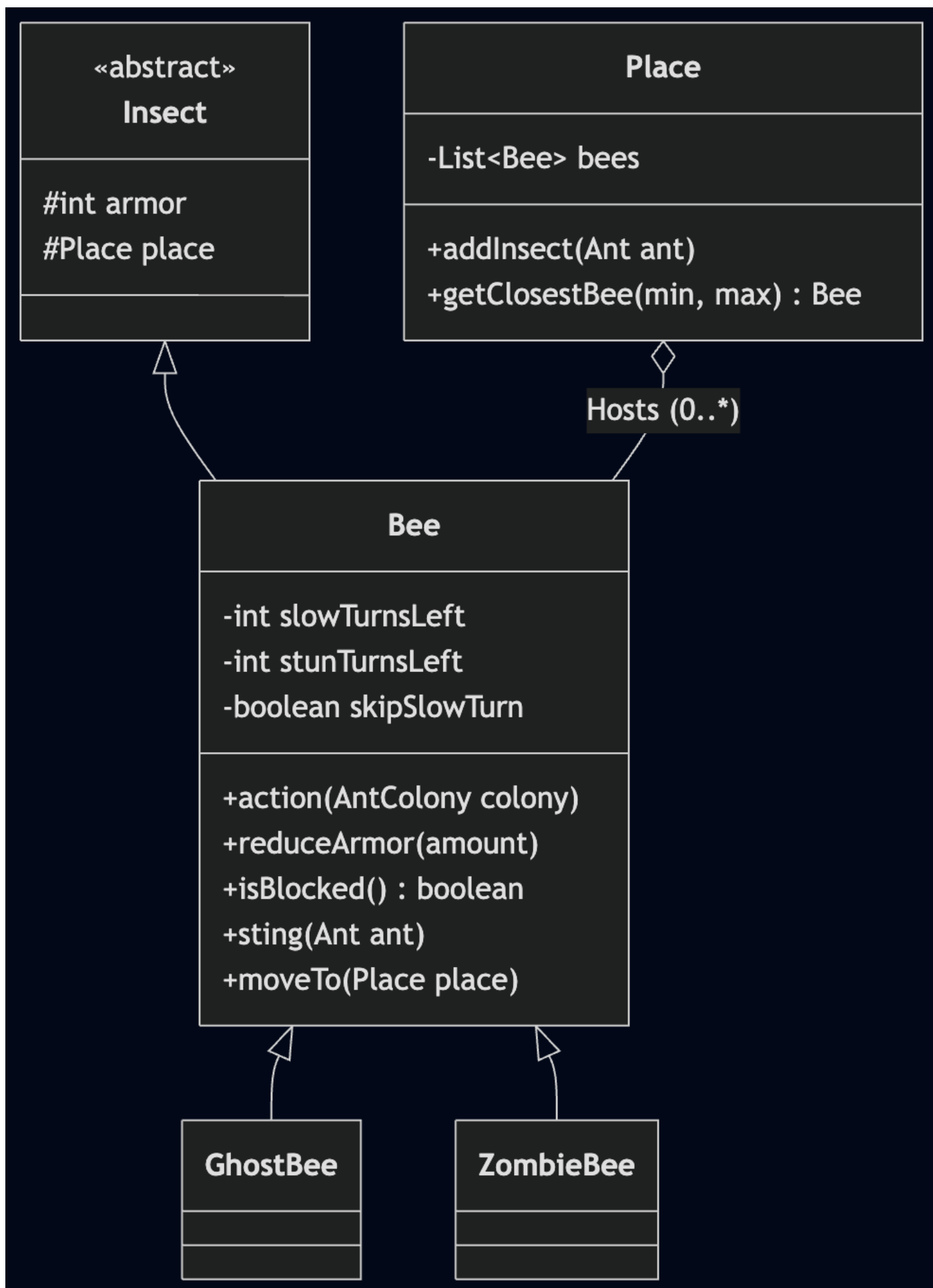
2. The Ant Inheritance Tree

This diagram focuses on the player's units. Everything inherits from the base `Insect` class, with `Ant` sitting above it adding player-specific stats like `foodCost`. The ranged attackers then branch off into their own `ThrowerAnt` hierarchy so they can share the targeting logic rather than each one duplicating it.



3. The Bee Hierarchy and Environment

This last diagram covers the enemies and the grid. `GhostBee` and `ZombieBee` both inherit from the main `Bee` class, and you can also see how `Place` fits in, representing the individual tiles on the board and how each one can hold multiple bees at once.



Scoring User Stories & tasks – Author: Paddy Costelloe

[1] - Scoring the user stories took place during sprint meetings and recorded on JIRA. This was to make the scoring a team effort and to ensure that no member of the team felt like they were being given too large of a workload. First, we would write up a basic one-line user story for JIRA and discuss what we thought would be involved in the tasks. The team would cover complexity and uncertainty based on what we had learned about the code given to us and our own coding experience.

Once the discussion was over, we would add the user story or task to the back log and assign it a developer. The team would usually try to keep the developers in certain areas. E.g. Parastoo for menus and UI, Shah for Audio and visual, Paddy for Gameplay features and documentation. But it was not always easy to maintain a single coder for each area due to team size and time restrictions.

After a developer was selected, the story or task would be given a score based on group consensus with the final vote given to the person doing the task. Each member of the team would vote in secret on each task using the Fibonacci sequence (1, 2, 3, 5, 8, 13, 21, 34) from one as a minimum to thirty-four as a maximum. Then we would publish them in team's chat and take the average. If someone put a wildly different number for a task to the other team members then it opened it back up to discussion as to why the number was so different. The benefit of the final score decision given to the developer who would be responsible for that area.

All items were given a score even if they were not stories, this included tasks, requests and bugs. This was to ensure that workloads were tracked. For any task, request or bug entered mid sprint the scoring was the responsibility to the creator but was passed by the scrum lead (Paddy) to ensure it was not too high or low compared to other current work.

Key	Summary	Work type	Epic	Status	Assignee	Story points
SCRUM-44	Create a Jira flow chart	Task	CREATE A FULLY FUNCT...	COMPLETE	PC	1
SCRUM-64	thrower ants and bodyguard branch test	Request	CREATE A FULLY FUNCT...	COMPLETE	PC	2
SCRUM-65	merge queen and thrower ants/bodyguard branches after testing	Request	CREATE A FULLY FUNCT...	COMPLETE	PC	3
SCRUM-66	test action reporting to thrower ants and bodyguard	Request	CREATE A FULLY FUNCT...	COMPLETE	PC	1
SCRUM-67	test hungry ant fix	Request	CREATE A FULLY FUNCT...	COMPLETE	PC	1
SCRUM-32	As a Developer, I want to give the player better control of the game, so that they ca...	Story	CREATE A FULLY FUNCT...	COMPLETE	PS	3
SCRUM-68	Create a Menu system that the rest of the team can add to.	Task	CREATE A FULLY FUNCT...	COMPLETE	PS	13
SCRUM-69	create game menu flowchart	Task	CREATE A FULLY FUNCT...	COMPLETE	PC	2
SCRUM-70	manage jira sprint 4	Task	CREATE A FULLY FUNCT...	COMPLETE	PC	1
SCRUM-79	Create a menu flow chart	Task	CREATE A FULLY FUNCT...	COMPLETE	PC	1
SCRUM-80	Create a UI layout	Task	CREATE A FULLY FUNCT...	COMPLETE		3
Work items completed outside of sprint						View in All work navigator
Key	Summary	Work type	Epic	Status	Assignee	Story points
SCRUM-41	As a Developer, I want Create a Ghost Bee that is hard to attack so that the Bee hiv...	Story	CREATE A FULLY FUNCT...	COMPLETE	SQ	2
SCRUM-40	As a Developer, I want Create Zombie Bee that attacks multiple ants at once so that...	Story	CREATE A FULLY FUNCT...	COMPLETE	SQ	3
SCRUM-37	As a Player I want to hear music , so that the game is more interesting as it is rather...	Story	CREATE A FULLY FUNCT...	COMPLETE	SQ	3

When the team first met, it was decided to build the backlog on Trello in the first instance for its simplicity and ease of use. However, as the group progressed through the back log, we discovered it lacked the granularity to correctly track sprints and progress being made on tasks without installing extra plugins. The team decided in week two to move the entire backlog to Jira for its more developer focused array of tools that fit more closely to the scrum methodology.

Agile Scrum Report

Weekly Sprint – Author: Paddy Costelloe

Due to the time limit for this project (around 8 weeks) a weeklong sprint period was agreed by the team instead of the more traditional 2-week sprints in industry. This would help the team stay on track and ensure that the sprints coincided with classes where the team would have a natural meeting point.

Each week on a Tuesday the team agreed to meet up and review the Jira boards for the last sprint. During this review the team would agree on what back log completion criteria was met as per the scrum guidance. Once completed task was reviewed, we would move onto incomplete tasks and what was holding up completion and any blockers the team were experiencing before moving the incomplete tasks into the next sprint.

Once blockers and completed tasks were covered, the team would move onto building the next sprint out of the backlog of items, new and incomplete. Incomplete items got reviewed and automatically included in the next sprint unless there was good reason to put them on hold.

The team used Jira's own task identifiers for the backlog which includes, Tasks, User Stories, Bugs, Requests and Features.

Tasks took the form of documentation, non-user-story coding, maintenance and the like.

User Stories and Features often fell into the same category of overall game features. Smaller game features were given stories while larger overarching features like a full menu were given full feature status. Unfortunately, there didn't appear to be a good way to make a user story a child of a feature without overcomplicating things for the small team.

Bugs are where the team recorded any bugs found and gave them to team members to fix depending on who was closest to the problem in their current or recent past task list.

Requests were generally used to ask other team members to test branches or features before merging them back to main.

Scoring the tasks (including user stories) was done using a range of the Fibonacci sequence (1, 2, 3, 5, 8, 13, 21, 34). In the meetings this was done secretly for each task and then revealed. Once revealed the team would discuss the task to come up with a consensus score based on the scores we secretly gave. This is discussed more in the User stories section above.

during mid sprint, each team member could also raise bug issues found on Jira and assign them to appropriate individuals, usually in line with whoever had worked on that area recently. For example, there was an issue with the music, whereby when the game was reset the music started playing again over the top of the last track. This could lead to multiple tracks being layered on top of each other completely breaking the audio. Since this had to do with the menu on which team member Parastoo had been working most recently, she was given the task of fixing the bug. In these cases, the bug fixer was given the option of scoring the task with the agreement of the scrum leader (Paddy). These scores were usually kept rather low unless the issue was completely game breaking and took a lot of time to fix. If that were the case, then it would be reviewed in the next sprint meeting.

Slack usage – Author: Paddy Costelloe

Generally speaking, slack was used for day-to-day communication and as a stand in for daily scrum stand ups. The team knew we would not be able to maintain the classic 15-minute standup at the beginning of each day due to other commitments and workloads. Instead, the team agreed to post daily diaries of activities even if the activity was just "no progress" and stay in open communication on slack to ask for help and report issues.

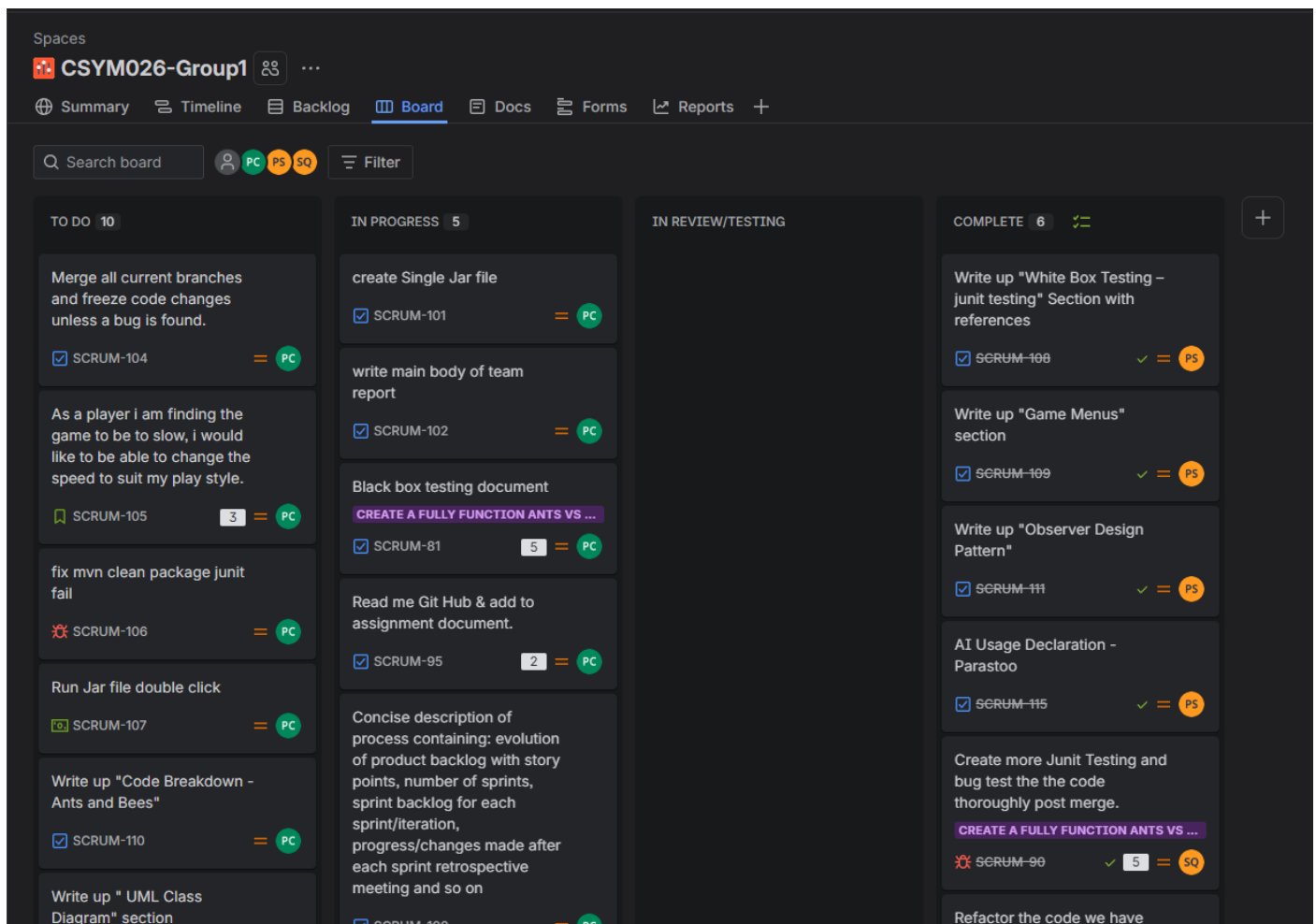


Figure 1 - board example Jira.

The Jira board still works very similarly to Trello with the added advantage of being tracked in the sprint sections for the backlog. In the backlog we could correctly track and score point, as this was not available in base Trello.

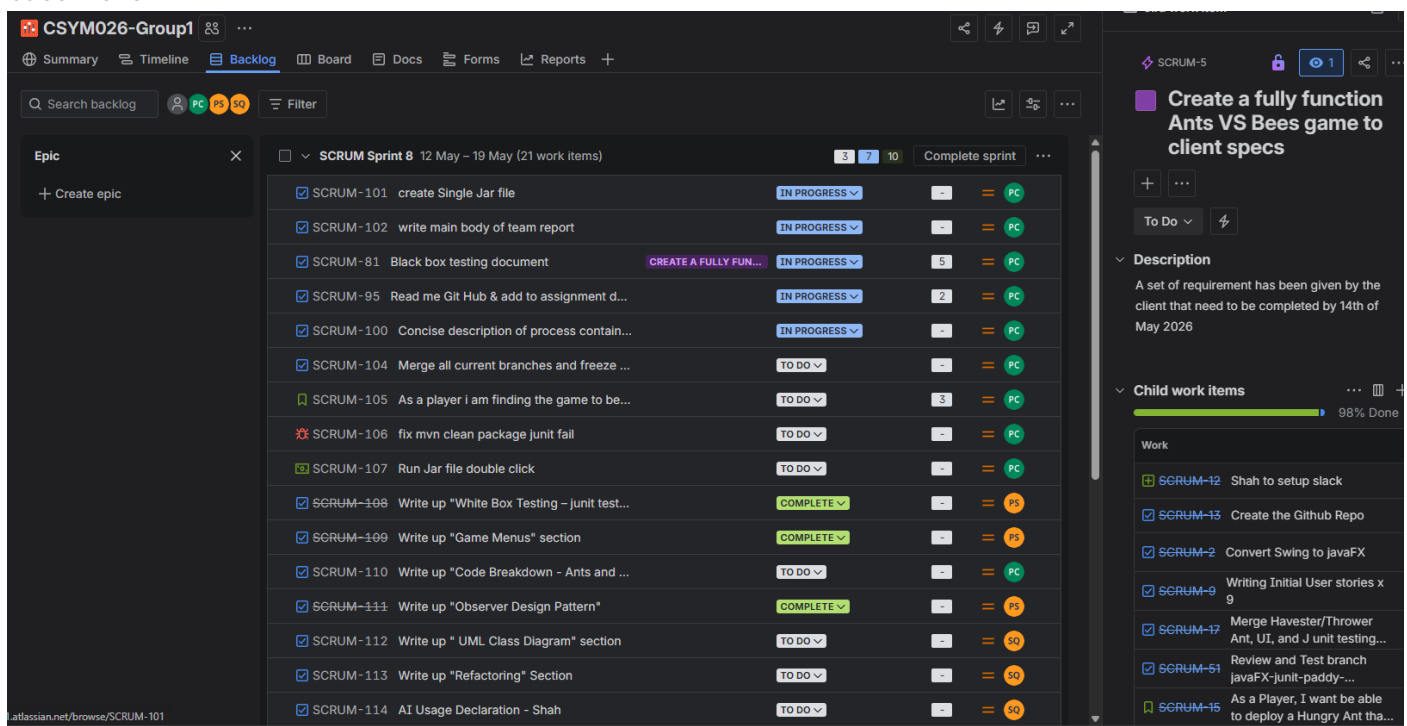


Figure 2 - back log example.

Agreed Workflow – Author: Paddy Costelloe

The bellow flow chart for fig-1 represents an idealised workflow for the team and was agreed with the team in meetings and over slack. This workflow would help each team member understand the commitment/completion criteria aimed for, with each task/user story.

Given more time and dedicated development time we feel this workflow would have ensured that each task is completed with the correct steps for meeting our scrum commitment.

In retrospect that flow chart would work best in a true company setting where each member of the team was working in the same office space or virtual space for the same time-period. However, due to other commitments and time issues this idealised workflow was used as a guide rather than a strict rule.

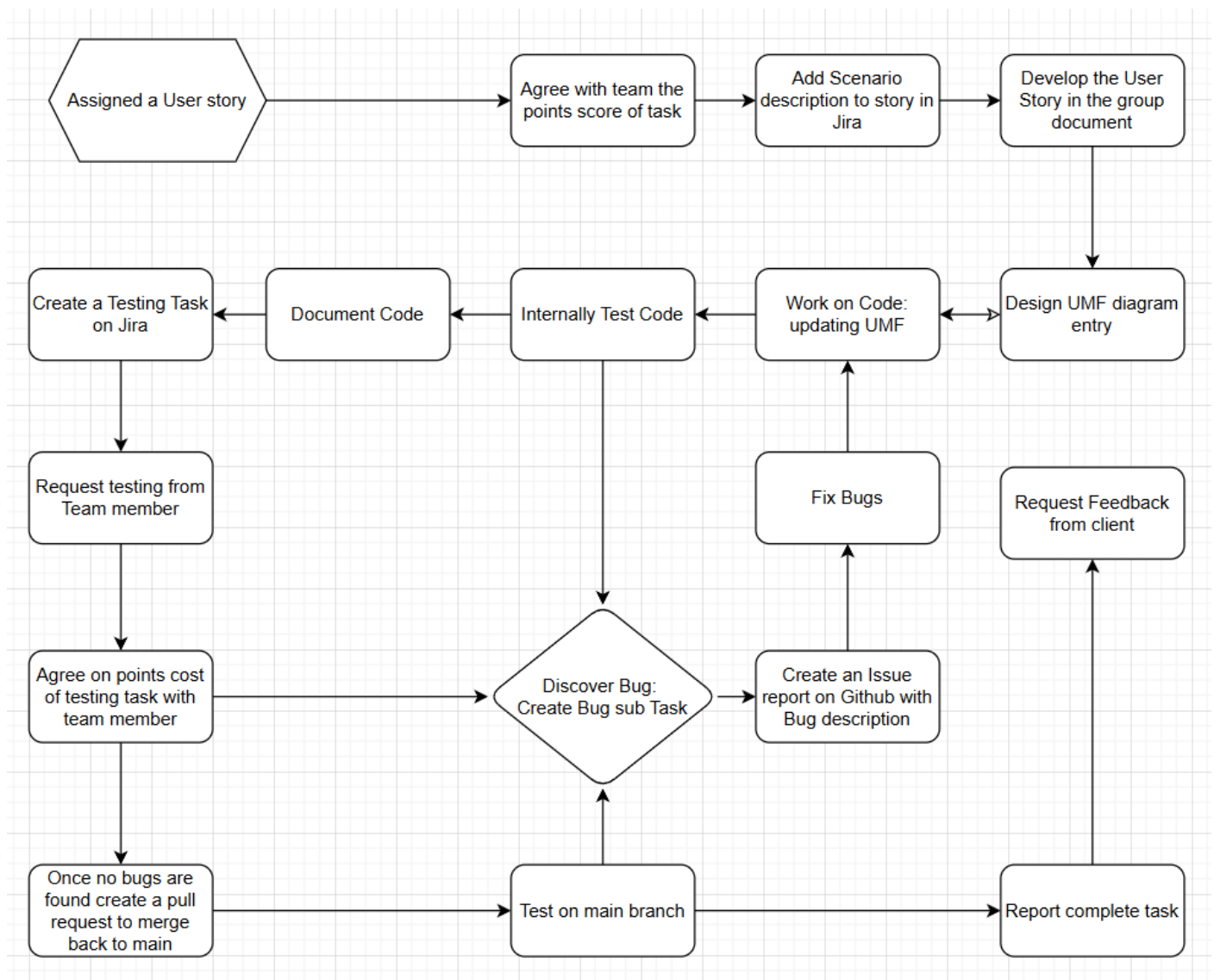


Figure 3- agreed workflow.

The team chose to focus on communication, documentation and testing over some of the other areas such as UMF design which came later.

The documentation helped the team understand where and when another member of the team had made a change and so helped the team in knowing who to talk to when they code needed understanding.

Example of code documentation:

```

/**
 * @author Parastoo , Date 19/04/2026
 *     Pauses the game by stopping the animation timer.
 *     This allows the menu system to temporarily freeze gameplay.
 */
public void pauseGame() {
    if (clock != null && isRunning && !isPaused) {
        clock.stop();
        isPaused = true;
    }
}
// Parastoo: code change ends here.

```

The above example shows how the team documented changes to code. Parastoo not only documents the method but also insets a “//Parastoo: code change ends here” comment to show where the edit ends.

Even if a change is made mid method, team members were encouraged to leave a comment with name and date for future developers to reference if they needed to change code. Over time these comments were deleted if they caused code bloat but it meant that the team knew what had been changed day to day and could track the changes back to git hub commits.

given that the code base given to the team was external code, this helped the team to not break the code with changes, especially as merges and commits where happening with the other developers and branches.

Client Review Process – Author: Paddy Costelloe

After each week’s main sprint meeting the team would meet with the client. This is a break from normal scrum workflow as usually the main sprint meeting would directly involve the client. But because of time constraints on the client and the team, the client gave feedback at the end of each sprint meeting after tasks and story points were reviewed.

If this had been a more traditional development environment the client would be involved at each stage with regular meetings and updates. This is the closest we could achieve while trying to maintain a scrum style methodology. This style of flexibility sits closer to the agile methodology while imitating scrum tenants but would not be considered full scrum in the industry according to [\[3\]](#) the scrum guidelines.

Managing GitHub and Jira – Author: Paddy Costelloe

Jira became the focal point of the teams’ efforts to work in an agile style using scrum as a basis. This meant meeting weakly to review the backlog setup on Jira and updating it throughout the week. Within Jira tasks, stories, bugs, features and requests could be directly assigned to each team member. Due to the size of the team, it was decided that we should stick to user the overarching tasks in the backlog and avoid using sub tasks. Though it is noted that this would be especially useful to the team if there were sub teams to parcel out larger epics and features. While it was not mandatory to keep tasks up to date on a day-by-day basis it was encouraged, to enable us to look at progress via the Scrum burn down reports.

While the reports were not always perfect due to late reporting by team members, they gave us a good idea each week in review of what was being completed and in what time frame.

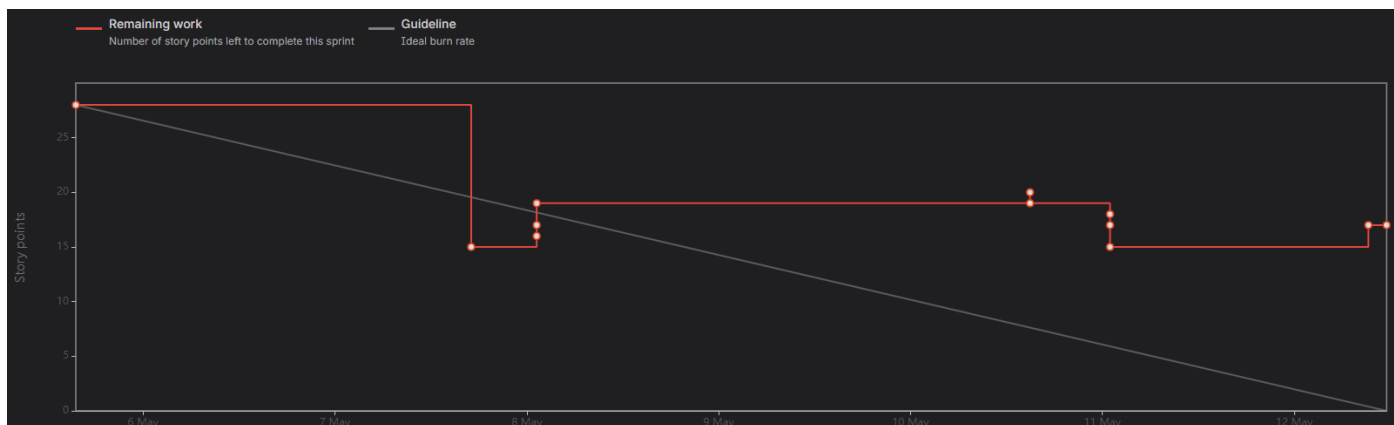


Figure 4 - Burn down report example sprint 7.

The Sprint report above shows that the team was working sporadically. This is correct as we all have external jobs and other projects as well as this project. It is important to note that the grey bar is the perfect sprint burn down to aim for. But the size of the project and its teams as well as team member external commitments effected the burn down. That said it is still good data and reviewed at each week to give the team some automated feedback on how well we were doing.

GitHub was used extensively to manage the project and segment code worked on. This was not always perfect so each week the team would go through a merge to get back the main on the Tuesday morning meeting. Any work incomplete at that stage would be moved back to the backlog and the branch would not merge. The aim was to have a working branch to show the client during the client review meeting. If a branch was likely to break, it would not be used. Each branch would be evaluated before and after merging. Even then the branch would not merge directly back into main. Instead, the scrum leader would create a “merge_to_main_date” branch to merge into and test before anything was aloud to touch the main default branch.

It became clear as the team was working that we needed to improve on our naming conventions as during the many branching and merging there was some confusion on which branch was still live. This is when adding developer names and dates to the branch name became best practice.

Sprints and retrospectives overview

In total we did eight sprints across the development period:

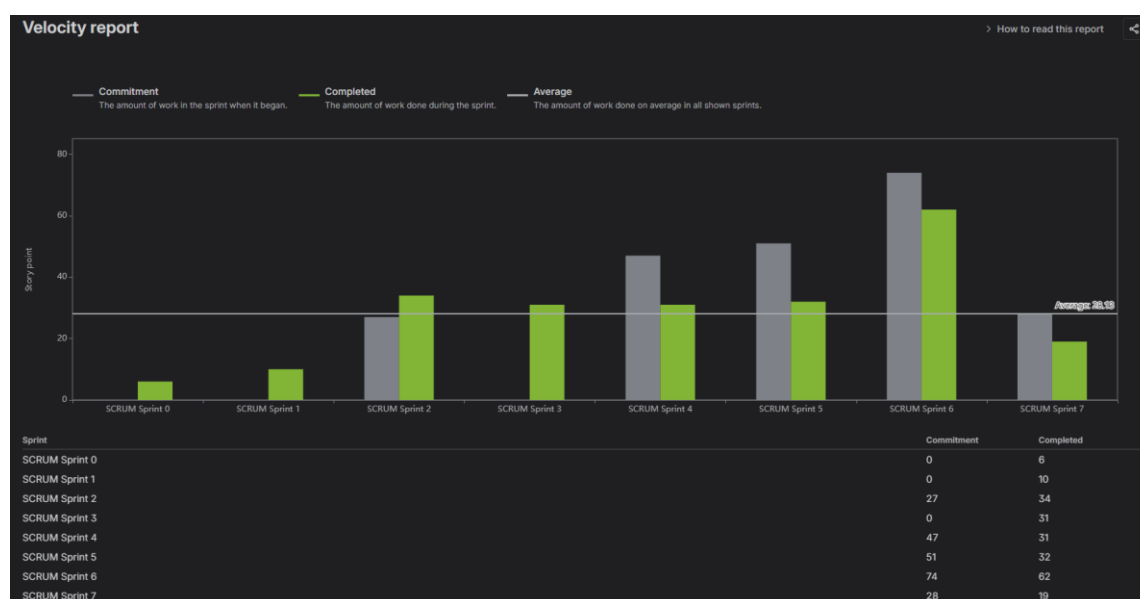


Figure 5 - sprint velocity report.

At the end of each sprint/week we had an hour-long meeting to discuss what went right and what went wrong in the previous sprint as well as discuss what was going to enter the current sprints backlog in the next sprint/week.

One improvement learned from our weekly meetings was that we needed a better way to keep the retrospective as Jira own document creation space was overly complicated. This meant sprint retrospectives were hard to access and a basic text document was easier to work with.

Sprint Retrospective – Sprint 0 (Ants vs Bees Group 1)

In attendance – Paddy & Parastoo

Absent - Shah

Date: 24 March 2026

What Went Well:

- Strong initial project setup including agreement on Agile (Scrum) methodology and working structure.
- Clear definition of team working agreements, including code ownership, author attribution, and safe-to-remove code practices.
- Early establishment of user stories and shared templates to ensure consistency.
- Clear task allocation for the first development cycle (ants implementation and testing setup).
- Effective communication and collaboration between attending team members.

What Didn't Go Well:

- Key team member absent, limiting full team input into early architectural decisions.
- Some uncertainty around tooling and environment setup (e.g. Visual Studio familiarity).
- Unclear initial approach to UI implementation (code vs Scene Builder) requiring discussion.
- Backlog and task tracking not fully populated, noting Trello limitations.
- Limited clarity on testing strategy and implementation sequencing at the outset.

What Could Be Improved:

- Ensure full team participation in initial planning sessions where possible to align decisions.
- Continue building and maintaining a clear product backlog (e.g. Trello/Jira) from the beginning.
- Standardise development approach early (e.g. UI decisions, coding style, architecture).
- Integrate testing (JUnit) alongside development rather than delaying it.
- Improve onboarding and setup support for tools and environments to reduce early friction.
- Maintain regular communication through agreed channels (Slack updates every day with diary channel).

Sprint Retrospective – Sprint 1 (Ants vs Bees group 1)

In attendance – Paddy & Parastoo

Absent - Shah

Date: 31 March 2026

What Went Well:

- Successful setup and use of Jira workflow (backlog, epics, stories, story points).
- Initial development progress including UI feedback system and JUnit setup.
- Strong communication with agreement on weekly scrums and daily Slack updates.
- Clear task allocation and ownership across team members.
- Effective adoption of Agile practices such as story pointing.

What Didn't Go Well:

- Multiple branches caused fragmentation and delayed integration.
- Technical issues (e.g. IDE problems) disrupted progress.
- Testing workflow was not fully clear or consistently applied.
- Missing team members limited full participation in planning.

What Could Be Improved:

- Maintain a single stable base branch earlier in the sprint.
- Clarify testing responsibilities and workflow.
- Encourage full team participation in sprint activities.
- Reduce technical blockers by preparing environments in advance.

Sprint Retrospective – Sprint 2 (Ants vs Bees Group 1)

In attendance – Paddy & Parastoo

Absent - Shah

Date: 7 April 2026

What Went Well:

- Strong development progress with multiple ant types completed (fire, ninja, wall, hungry, scuba).
- Effective use of Jira for tracking tasks, story points, and collaboration decisions.
- Clear task allocation for the next sprint, with ownership assigned to each team member.
- Active communication via Slack and meetings, including discussion of bugs and workflow improvements.
- Good use of Agile practices such as story pointing (Fibonacci estimates).

What Didn't Go Well:

- Branching and merging remained incomplete, with features developed but not fully integrated.
- Testing was informal and inconsistent (mainly quick manual checks).
- One team member was absent from the meeting, limiting full collaboration.

What Could Be Improved:

- Improve branch management by merging work into a single main branch more regularly.
- Establish clearer and more consistent testing practices.
- Ensure all team members attend meetings or provide structured input beforehand.
- group logging bugs systematically in Jira.

Sprint Retrospective – Sprint 3 (Ants vs Bees Group 1)

In attendance – Paddy & Parastoo

Absent - Shah

Date: 14 April 2026

What Went Well:

- Significant progress with many ant and bee features completed or near completion.
- Successful implementation of an action reporting system to improve visibility of gameplay events.
- Improved backlog management with reduced number of outstanding tasks.
- Clear planning of deadlines, including submission timeline.
- Effective task allocation with defined responsibilities for each team member.

What Didn't Go Well:

- Many features still required testing and merging before being fully complete.
- Game balancing issues identified, with gameplay becoming too easy.
- Large tasks (e.g. menu system, refactoring) not yet clearly divided or scoped.
- Absence of a team member reduced input for key architectural decisions.
- Some coordination challenges around branch integration and testing responsibilities.

What Could Be Improved:

- Prioritise regular merging and testing to ensure all features are integrated smoothly.
- Improve game balance and overall gameplay experience through tuning and refactoring.
- Break down large tasks (e.g. menu system, refactoring) into smaller, manageable items.
- Ensure full team participation in meetings to support collaborative decision-making.

Sprint Retrospective – Sprint 4 (Ants vs Bees Group 1)

In attendance – Paddy & Parastoo

Absent - Shah

Date: 21 April 2026

What Went Well:

- Continued development progress with UI improvements such as mouse hover feedback and play/pause - functionality.
- Identification and replication of bugs across branches improved shared understanding of issues.
- Collaborative problem-solving during the session (e.g. testing, debugging environment issues).
- Good visibility of system behaviour through UI and action reporting improvements.

What Didn't Go Well:

- Multiple failing tests and inconsistencies across development environments (SDK/configuration issues).
- Incomplete or buggy implementations (e.g. Bodyguard Ant behaviour, deployment issues).
- Integration issues when working with other team members' code.
- Some planned features (e.g. score system) not completed within the sprint.
- Recurring absence of a team member impacting visibility of progress and validation of work.

What Could Be Improved:

- Focus more time on testing, debugging, and stabilising existing features before adding new ones.
- Standardise development environments (SDKs, build configs) to reduce inconsistent results.
- Ensure bugs are clearly documented and tracked in Jira for follow-up.
- Encourage consistent team participation to improve collaboration and decision-making.

Sprint Retrospective – Sprint 5 (Ants vs Bees Group 1)

In attendance – Paddy, Parastoo & Shah

Date: 28 April 2026

What Went Well:

- Core gameplay systems are now largely functional, with stable builds achieved from merged team branches.
- Successful completion of key features such as Bodyguard Ant, Helpdesk tooltips, and player data framework.
- Unit testing improvements, including fixing failing Hungry Ant tests and achieving consistent test passes.
- Improved usability through UI enhancements (mouse-over tooltips and interaction feedback).

What Didn't Go Well:

- Merge conflicts with Shah's branch caused functional issues (e.g. handle-click and selection bugs).
- Limited visibility of one team member's work due to absence from meetings such as UML diagrams.
- Some key features only partially completed (e.g. achievements, scoring system, full integration).
- Growing technical complexity made coordination and integration more difficult.

What Could Be Improved:

- Improve merge strategy further by maintaining small, isolated branches and frequent integration.
- Increase communication across all team members to ensure visibility and alignment.
- Focus remaining effort on stability, debugging, and integration rather than adding new mechanics.
- Ensure all required deliverables (UML, game difficulty, documentation) are consistently reviewed and demonstrated.

Sprint Retrospective – Sprint 6 (Ants vs Bees Group 1)

In attendance – Paddy & Parastoo

Absent - Shah

Date: 05 May 2026

What Went Well:

- Successful completion and integration of major features including UI, sound effects, game difficulty, and tutorial.
- End-game scoring, player tracking, and feedback systems fully implemented and working.
- Critical bugs (e.g. handle-click issue) were identified and resolved.
- All team members were present, improving communication and visibility of progress.
- Clear transition from development into final phase of testing and submission preparation.

What Didn't Go Well:

- JUnit testing coverage remained insufficient and required additional work.
- White-box testing documentation was still incomplete at the end of the sprint.
- Refactoring posed risks of breaking working code and had to be limited.
- Time pressure increased as the project approached final submission.
- Some minor bugs still present (e.g. music duplication on reset).

What Could Be Improved:

- Maintain consistent automation of testing alongside feature development via JUnit testing.
- Improve time management across modules to reduce end-of-project pressure.
- Ensure documentation (README, reports, testing) is started earlier and built incrementally.
- Adopt strict bug-tracking workflows (Jira) with clear ownership and quick resolution cycles.

No notes were taken during sprint 7 as it was a live demo with the client.

Code Breakdown

Ants and Bees classes – Author: Parastoo Shojaei Sarcheshmeh

Overview – Author: Parastoo Shojaei Sarcheshmeh

The game is built around a layered inheritance hierarchy that separates shared behaviour from specialist behaviour. At the top sits the abstract `Insect` class, which every creature in the game extends. Below it, `Ant` and `Bee` each add their own logic, and below them sit all the concrete unit types such as `ThrowerAnt`, `HungryAnt`, `GhostBee`, and `ZombieBee`.

The Insect Base Class – Author: Parastoo Shojaei Sarcheshmeh

`Insect.java` is the root of the hierarchy. It holds the fields and methods that every creature shares: `armour`, `place`, `isWaterSafe`, `insectName`, `descriptor`, and `report`. It also defines two abstract methods that every subclass must implement:

```
public abstract void leavePlace();
public abstract void action(AntColony colony);
```

The `reduceArmor()` method handles damage and automatically calls `leavePlace()` when armour reaches zero. This means any subclass that wants custom death behaviour simply overrides `reduceArmor()`. The `Bee` class uses exactly this approach to trigger score tracking, and `FireAnt` uses it to trigger its explosion.

The `setReport()` and `getReport()` methods give every insect a live status string that the colony and hive report panels display each turn.

The Ant Abstract Class – author: Parastoo Shojaei Sarcheshmeh

Ant extends Insect and adds fields specific to player-deployed units: foodCost, damage, and isBuffed. It provides two constructors so subclasses can set armour and food cost in a single super call.

Four methods support the BodyguardAnt containment system. The default implementations in Ant return safe fallback values:

```
public boolean canContainAnt() {
    return false;
}
public boolean canBeContained() {
    return true;
}

public void containAnt(Ant ant) {
}
public Ant getContainedAnt() {
    return null;
}
```

BodyguardAnt overrides all four. The blocksPath() method defaults to true, meaning bees stop and attack any ant they encounter. NinjaAnt overrides this to return false, allowing bees to walk straight through it while still being damaged.

Ant Subclasses – Author: Parastoo Shojaei Sarcheshmeh

HarvesterAnt is the simplest subclass. Its action() calls colony.increaseFood(1) once per turn, giving the player resources to deploy more units.

ThrowerAnt introduces ranged combat. Its getTarget() method calls place.getClosestBee(0, 3), targeting any bee within three places. ShortThrowerAnt and LongThrowerAnt both extend ThrowerAnt and override only getTarget() to change the valid range:

```
@Override
public Bee getTarget() {
    return place.getClosestBee(0, 2);
}

@Override
public Bee getTarget() {
    return place.getClosestBee(4, Integer.MAX_VALUE);
}
```

ScubaThrowerAnt extends ThrowerAnt and simply calls setIsWaterSafe(true) in its constructor, inheriting all throwing behaviour while gaining the ability to deploy on water tiles.

SlowThrowerAnt and StunThrowerAnt extend ThrowerAnt but override action() entirely. Instead of dealing damage, they call target.applySlow(3) or target.applyStun(1) on the target bee, applying status effects that alter the bee's turn behaviour.

WallAnt overrides action() with an empty body. Its value comes entirely from its high armor of 4, making it a pure defensive unit that simply absorbs damage.

FireAnt overrides `reduceArmor()`. When its armor reaches zero, it saves a reference to its current Place, calls `super.reduceArmor()` to trigger removal, and then loops over all bees in the saved place dealing 3 damage each:

```
Place currentPlace = this.getPlace();
super.reduceArmor(amount);

if (this.getArmor() <= 0 && currentPlace != null) {
    Bee[] bees = currentPlace.getBees();
    List<Bee> targets = new ArrayList<>(Arrays.asList(bees));

    for (Bee bee : targets) {
        bee.reduceArmor(3);
    }
}
```

NinjaAnt overrides both `blocksPath()` and `action()`. Its `action()` loops over all bees in its current tile and reduces each by 1 armor, making it effective against groups. Since `blocksPath()` returns false, bees never stop to attack it directly.

HungryAnt uses a `digestTimeLeft` counter to manage a two-phase cycle. When ready, its `action()` calls `eatBee()`, which removes the bee from the board, adds a kill to `ScoreManager`, and sets `digestTimeLeft = 3`. JavaFX interaction inside `eatBee()` is handled via reflection so the class can be unit tested without a running UI:

```
public void eatBee(Bee bee) {
    if (bee != null) {
        bee.leavePlace();
        core.ScoreManager.addKill(bee);
        try {
            Class<?> gameClass = Class.forName("core.AntGame");
            java.lang.reflect.Field instanceField = gameClass.getField("instance");
            Object gameInstance = instanceField.get(null);
            if (gameInstance != null) {
                java.lang.reflect.Method method =
gameClass.getMethod("removeBeeInstantly", Bee.class);
                method.invoke(gameInstance, bee);
            }
        } catch (Exception e) {
        }

        this.digestTimeLeft = 3;
    }
}
```

BodyguardAnt overrides `canContainAnt()` to return true and stores the contained ant in a private field. Its `reduceArmor()` saves the current Place reference before calling `super.reduceArmor()`. If the bodyguard dies and still holds a contained ant, it calls `currentPlace.addInsect(containedAnt)` to restore the protected unit to the tile.

The Bee Class – Author: Parastoo Shojaei Sarcheshmeh

Bee extends `Insect` and adds `slowTurnsLeft`, `skipSlowTurn`, and `stunTurnsLeft` fields to manage status effects. Its `action()` checks stun first, then slow, then either stings a blocking ant or moves forward:

```
public void action(AntColony colony)
{
```

```

    if(stunTurnsLeft > 0)
    {
        stunTurnsLeft--;
        setReport("armor = " + armor + " action: Stunned");
        return;
    }
    if(slowTurnsLeft > 0)
    {
        slowTurnsLeft--;

        if(skipSlowTurn)
        {
            skipSlowTurn = false;
            setReport("armor = " + armor + " action: Slowed - skipped turn");
            return;
        }
        else
        {
            skipSlowTurn = true;
        }
    }

    if(this.isBlocked())
        sting(this.place.getAnt());
    else if(this.armor > 0)
        this.moveTo(this.place.getExit());
}

```

Bee also overrides `reduceArmor()` to call `ScoreManager.addKill(this)` before calling `leavePlace()`, ensuring every bee death is scored regardless of which ant dealt the killing blow.

`isBlocked()` was updated to check whether that ant's `blocksPath()` returns true, enabling the `NinjaAnt` to coexist with bees without triggering combat:

```

public boolean isBlocked()
{
    return this.place.getAnt() != null && this.place.getAnt().blocksPath();
}

```

GhostBee overrides `reduceArmor()` with a 50% evasion mechanic:

```

@Override
public void reduceArmor(int amount) {
    double dodgeChance = Math.random();
    if (dodgeChance < 0.5) {
        this.setReport("turned intangible and dodged the attack!");
    } else {
        super.reduceArmor(amount);

        if (this.getArmor() > 0) {
            this.setReport("was hit and has " + this.getArmor() + " armor left.");
        } else {
            this.setReport("was vanquished!");
        }
    }
}

```

```

    }
}

```

ZombieBee overrides `action()` to attack both the current tile and the exit tile simultaneously, implementing area-of-effect damage. If no ants are present it moves normally.

The Place Class – Author: Parastoo Shojaei Sarcheshmeh

Place acts as the spatial node of the game. Each place holds a single Ant reference and an ArrayList of Bee objects. `addInsect(Ant ant)` was extended to support the BodyguardAnt containment system:

```

public void addInsect(Ant ant)
{
    if(this.ant == null)
    {
        this.ant = ant;
        ant.setPlace(this);
    }
    else if(this.ant.canContainAnt() && this.ant.getContainedAnt() == null &&
ant.canBeContained())
    {
        this.ant.containAnt(ant);
        ant.setPlace(this);
    }
    else if(ant.canContainAnt() && ant.getContainedAnt() == null &&
this.ant.canBeContained())
    {
        ant.containAnt(this.ant);
        this.ant = ant;
        ant.setPlace(this);
    }
    else
    {
        System.out.println("Already an ant in " + this);
    }
}

```

`removeInsect(Ant ant)` was similarly extended. If the ant being removed holds a contained ant, the contained ant is promoted to the primary position.

`getClosestBee(int minDistance, int maxDistance)` walks the entrance chain from the ant's tile outward and returns the first bee found within the given range, enabling all range-based targeting behaviour.

The AntColony Class – Author: Parastoo Shojaei Sarcheshmeh

AntColony constructs the tunnel grid as a linked list of Place objects. Water tiles are placed randomly using a seeded Random object so the same seed always produces the same layout. `deployAnt()` validates food cost, water safety, and queen singleton rules before calling `place.addInsect()`. `queenHasBees()` checks both the queenPlace and the colonyBase reference, so losing either endpoint ends the game correctly.

The Hive Class – Author: Parastoo Shojaei Sarcheshmeh

Hive extends Place and stores a `Map<Integer, Bee[]>` mapping turn numbers to bee waves. `invade()` is called each turn and moves the wave for the current turn number into random tunnel entrances.

makeRandomLevelHive() uses a seeded Random to generate a mix of standard, ghost, and zombie bees proportional to the selected difficulty level, ensuring higher difficulties send more bees with higher armor values.

Design Pattern: Observer Pattern – Mouse Event Systems

Singleton – QueenAnt – Author: Paddy Costelloe

For the Queen Ant, only one version of the queen should ever exist, so the singleton design pattern was implemented to ensure that a single instance can only be created once. The Bill Pugh lazy initialization method, meaning that the instance is not created until the first call of the method getInstance() which in turn calls the QueenAntSingletonHelper class. Once the inner class is created it holds a single static final variable of QueenAnt and will always return the same instance after the first call. [2]

```
public class QueenAnt extends ScubaThrowerAnt {
    /**
     * @author Paddy 11/05/2026
     * This constructor is set to private so it can only be called by the
     * QuennAntSingletonHelper method to
     * return only one instance of QueenAnt once initialized.
     * it sets all the standar ant variables.
     */
    //Bill Pugh Singleton method
    //QueenAnt queenAnt = QueenAnt.getInstance();
    private QueenAnt() {
        super();
        armor =1;
        foodCost = 6;
        damage = 1;
        insectName = "QueenAnt";
        setIsWaterSafe(true);
        System.out.println("Singleton instance created!");
        descriptor = "\nThe Queen Ant is the most important Ant in the colony.\nIf she
dies, it's game over!\nThe Queen Ant cannot be moved once placed.\nThere can only be one
true queen ant.\nThe queen ant has " + armor + " armor and costs " + foodCost + " food.";
        setReport("Armor = " + armor + " Action Taken: None");
    }
    /**
     * @author Paddy 11/05/2026
     * initialises a final static instance of the class QueenAnt contained in this inner
     class and re-refers to it if it is called again.
     */
    private static class QueenAntSingletonHelper {
        private static final QueenAnt INSTANCE = new QueenAnt();
    }
    /**
     * @author Paddy 11/05/2026
     * @return an instance of QueenAnt created by the QueenAntSingletonHelper method.
     * since the method refers to an instance, once created the same instance is return
     with each access ensuring only one Queen.
     */
    public static QueenAnt getInstance() {
        return QueenAntSingletonHelper.INSTANCE; // Triggers QuennAntSingletonHelper
class load
    }
```

```
}
```

Observer pattern – Author: Parastoo Shojaei Sarcheshmeh

The Observer pattern is a behavioural design pattern in which an object (the subject) maintains a list of dependents (observers) and notifies them automatically when its state changes [\[6\]](#). In this project, the Observer pattern appears in the mouse event system inside `AntGame.java`. JavaFX's built-in event handling acts as the subject, and the game's UI components function as the observers that react to mouse input.

How It Is Applied

In `AntGame.java`, two mouse event handlers are registered on the game pane during construction:

```
this.setOnMousePressed(this::handleClick);  
this.setOnMouseMoved(this::handleMouseOver);
```

These two lines register `handleClick` and `handleMouseOver` as observer methods. JavaFX internally maintains a list of these listeners and calls them automatically whenever the user interacts with the game canvas [\[7\]](#). The game never polls for input; instead it simply reacts when notified.

`handleMouseOver` – The Observer in Action

`handleMouseOver` is the clearest example of the pattern. Every time the mouse moves, JavaFX fires a `MouseEvent` and notifies `handleMouseOver`. The method then checks which game element is under the cursor and updates the relevant UI panel:

```
private synchronized void handleMouseOver(MouseEvent e) {  
  
    double x = e.getX() / normalizedWidth;  
    double y = e.getY() / normalizedHeight;  
  
    mouseOverToolTip.setLayoutX(x + 10);  
    mouseOverToolTip.setLayoutY(y);  
  
    for (Rectangle rect : antSelectorAreas.keySet()) {  
        if (rect.contains(x, y)) {  
            mouseOverToolTip.setVisible(true);  
            Ant ant = antSelectorAreas.get(rect);  
            if (ant != null) {  
                mouseOverToolTip.setText(ant.getInsectName());  
                setFeedbackAntDescriptor(  
                    "Mouse over Ant to show description. \nAnt Description: \n" +  
                    ant.getDescriptor());  
                return;  
            }  
        }  
    }  
  
    for (Map.Entry<String, Rectangle> entry : beeSpawnAreas.entrySet()) {  
        if (entry.getValue().contains(x, y)) {  
            mouseOverToolTip.setVisible(true);  
            switch (entry.getKey()) {  
                case "Bee":  
                    mouseOverToolTip.setText("Bee");  
                    setFeedbackBeeDescriptor(  

```

```

        "Mouse over Bee to show description. \nBee Description:
\n" + Bee.getBeeDescriptor());
        break;
        case "Ghost Bee":
            mouseOverToolTip.setText("Ghost Bee");
            setFeedbackBeeDescriptor("Mouse over Bee to show description.
\nBee Description: \n"
                                   + GhostBee.getGhostBeeDescriptor());
            break;
        case "Zombie Bee":
            mouseOverToolTip.setText("Zombie Bee");
            setFeedbackBeeDescriptor("Mouse over Bee to show description.
\nBee Description: \n"
                                   + ZombieBee.getZombieBeeDescriptor());
            break;
        default:
            System.out.println(entry.getKey() + "got here 4");
    }
    return;
}
}
mouseOverToolTip.setVisible(false);
}

```

The mouse coordinates are first scaled by `normalizedWidth` and `normalizedHeight` to account for window resizing. The method then iterates over two maps — `antSelectorAreas` which maps clickable rectangles to `Ant` objects, and `beeSpawnAreas` which maps strings to rectangles — and checks whether the cursor falls inside any of them. When a match is found, the corresponding descriptor panel is updated and a tooltip is shown. When no match is found, the tooltip is hidden.

handleClick – A Second Observer

`handleClick` follows the same structure. JavaFX notifies it on every mouse press, and it determines whether the user clicked an ant selector, a tunnel tile, or the remover button:

```

private synchronized void handleClick(MouseEvent e) {
    if (!isRunning) {
        startGameLoop();
    }

    double x = e.getX() / normalizedWidth;
    double y = e.getY() / normalizedHeight;

    for (Rectangle rect : colonyAreas.keySet()) {
        if (rect.contains(x, y)) {
            Place targetPlace = colonyAreas.get(rect);
            if (selectedAnt == null) {
                feedbackUI = colony.removeAnt(targetPlace);
                return;
            } else {
                Ant deployable = buildAnt(selectedAnt.getClass().getName());

                if (targetPlace.getAnt() != null) {
                    Ant existingAnt = targetPlace.getAnt();
                    boolean validSharedPlacement = (existingAnt.canContainAnt()

```

```

        && existingAnt.getContainedAnt() == null &&
deployable.canBeContained())
        || (deployable.canContainAnt() &&
deployable.getContainedAnt() == null
        && existingAnt.canBeContained());
    if (!validSharedPlacement) {
        setFeedbackUI("Cannot deploy: Tile is already occupied!",
"error");
        playSoundEffect("error.wav");
        return;
    }
}
String result = colony.deployAnt(targetPlace, deployable);

if (result.toLowerCase().contains("success")) {
    setFeedbackUI(result, "success");
    playSoundEffect("pop.wav");
} else {
    setFeedbackUI(result, "error");
    playSoundEffect("error.wav");
}
}
return;
}
}
}

```

On the first click the game loop starts. On subsequent clicks the method checks the colony tile map, the ant selector map, and the remover area in sequence, responding to whichever element was clicked.

Coordinate Normalisation

Both handlers divide raw mouse coordinates by `normalizedWidth` and `normalizedHeight`. These two values are maintained by listeners registered on the stage in the constructor:

```

primaryStage.widthProperty().addListener((obs, oldVal, newVal) -> {
    normalizedWidth = (double) newVal / SCENE_WIDTH;
});
primaryStage.heightProperty().addListener((obs, oldVal, newVal) -> {
    normalizedHeight = (double) newVal / SCENE_HEIGHT;
});

```

These property listeners are also an application of the Observer pattern — they observe the stage's width and height properties and update the scale factors whenever the window is resized, keeping click and hover detection accurate at any window size [\[8\]](#).

The Tooltip Observer

A `mouseoverTooltip` `TextArea` was added as a floating label that follows the cursor. Its position is updated inside `handleMouseOver` on every mouse move event, making it a direct visual observer of the mouse position:

```

mouseoverTooltip.setLayoutX(x + 10);
mouseoverTooltip.setLayoutY(y);

```

Its bindings are unbound after initialisation so its position can be set freely:


```
initTextAreas(mouseOverToolTip,0,0,150,50,14,"rgb(76, 179, 231);", "tooltip");
mouseOverToolTip.layoutXProperty().unbind();
mouseOverToolTip.layoutYProperty().unbind();
```

Refactoring – Author: Shah Abdul Qadir

Migrating from Swing to JavaFX – Author: Shah Abdul Qadir

The starter code we got was built on legacy Java Swing. I knew right away that would be an issue. Swing is outdated and makes graphics feel incredibly clunky. To get things running smoothly and support Parastoo's new UI menus, I took the lead on moving the entire core game engine over to JavaFX.

Building a Custom Game Loop The old setup tied game turns and rendering to basic Swing timers. Those are famous for lagging and causing visual stutters, especially when the screen gets busy. My fix was to build a custom game loop using JavaFX's `AnimationTimer`. This ties straight into the screen's refresh rate, letting it run at about 60 frames per second, standard for modern games.

But we couldn't have the actual game rules running that fast, or the ants would sprint across the board. So, I split the game logic from the visual framerate. I put a time throttle inside the timer to lock the `update()` and `render()` methods at a steady 30 FPS.

```
clock = new AnimationTimer() {
    private long lastUpdate = 0;

    @Override
    public void handle(long now) {
// Run the game at roughly 30 FPS
// 33_333_333 nanoseconds is about 33.3 milliseconds
        if (now - lastUpdate >= 33_333_333) {
            update();
            render();
            lastUpdate = now;
        }
    }
}
```

Because of this, animations like thrown leaves and flying bees glide smoothly instead of just teleporting between tiles. It made the whole thing feel like a real game rather than a basic grid.

Thread Safety and Concurrency – Author: Shah Abdul Qadir

Switching to JavaFX meant dealing with strict threading rules. JavaFX demands that any screen updates happen on the main Application Thread. While testing, I noticed the game would completely crash with an `IllegalStateException` if a bee reached the queen and the background logic tried to instantly load the Game Over screen.

To solve this, I used `Platform.runLater()`. This tells Java to wait a split second and safely hand the screen update back to the main UI thread.

```
public void triggerGameOver() {
// Stop the game timers first
    if (clock != null) {
        clock.stop();
    }
    GameTimer.stop();
}
```

```
// Run any background save data stuff
    if (onGameEnd != null) {
        onGameEnd.run();
    }
// Pass control back to the UI thread so the screen can change safely without crashing
    if (onGameOver != null) {
        Platform.runLater(() -> onGameOver.accept(false));
    }
}
```

I had to do the exact same thing for the combat effects. When an ant attacks, a temporary leaf or splash image loads. To stop the game from freezing while that image loads into memory, I wrapped it in that same thread-safe method, so it processes safely in the background:

```
public void playTemporaryEffect(Image effectImage, double x, double y) {
    Platform.runLater(() -> {
        ImageView effectView = new ImageView(effectImage);
        effectView.setLayoutX(x);
        effectView.setLayoutY(y);
// Add the image to the screen
        gamePane.getChildren().add(effectView);
// Call a helper method to remove it after half a second
        removeEffectAfterDelay(effectView, 500);
    });
}
```

Dynamic UI Scaling – Author: Shah Abdul Qadir

The original code relied on hardcoded pixel coordinates. If a tile was at pixel 100, it stayed there. This meant the board wouldn't resize if you shrank the window or played on a smaller screen, it would just get cut off. Since we upgraded to a 1920x1080 layout, I had to build a dynamic scaling system.

Instead of doing the math for every single tile and bug on every frame, I set up a scale multiplier. I added listeners to the main window that track the width and height, updating the multipliers whenever the user resizes the window.

```
// Listen for window width changes
primaryStage.widthProperty().addListener((obs, oldVal, newVal) -> {
    normalizedWidth = (double) newVal / SCENE_WIDTH;
});
// Listen for window height changes
primaryStage.heightProperty().addListener((obs, oldVal, newVal) -> {
    normalizedHeight = (double) newVal / SCENE_HEIGHT;
});
```

Then, inside the render() method, I just scale the whole drawing canvas before drawing anything. It acts kind of like zooming a camera in or out.

```
gc.save();
// Apply the math multipliers to the whole canvas
gc.scale(normalizedWidth, normalizedHeight);
// Draw the game normally
drawBackground(gc);
drawAnts(gc);
```

```
drawBees(gc);  
gc.restore();
```

To keep mouse clicks accurate on a scaled screen, I updated the click listeners to divide the mouse position by the scale factor. If the screen shrinks by half, I divide the mouse coordinates by half. This maps your cursor perfectly back to the logical grid.

Graphics Integration – Author: Shah Abdul Qadir

I really wanted to ditch the boring placeholder shapes. I updated the graphics pipeline to render actual sprite sheets and animated GIFs so the game looked professional.

We had specific images for enemies like the Ghost Bee and Zombie Bee. I changed the drawing method to check the exact class type of each bee using instanceof, ensuring the right image popped up on the board.

```
private void drawBees(GraphicsContext gc) {  
    for (Map.Entry<Bee, AnimPosition> entry : allBeePositions.entrySet()) {  
        Bee bee = entry.getKey();  
        AnimPosition pos = entry.getValue();  
        // Draw specific images based on the class type  
        if (bee instanceof GhostBee && GHOST_BEE_IMAGE != null) {  
            gc.drawImage(GHOST_BEE_IMAGE, pos.x, pos.y, BEE_IMAGE_WIDTH,  
                ANT_IMAGE_HEIGHT);  
        } else if (bee instanceof ZombieBee && ZOMBIE_BEE_IMAGE != null) {  
            gc.drawImage(ZOMBIE_BEE_IMAGE, pos.x, pos.y, BEE_IMAGE_WIDTH,  
                ANT_IMAGE_HEIGHT);  
        } else if (BEE_IMAGE != null) {  
            // Default normal bee image  
            gc.drawImage(BEE_IMAGE, pos.x, pos.y, BEE_IMAGE_WIDTH,  
                ANT_IMAGE_HEIGHT);  
        }  
    }  
}
```

I also built an image caching system. Loading heavy images straight from the hard drive every turn was causing massive lag spikes. By pre-loading all the graphics into RAM right when the game starts, the JavaFX thread never stalls during combat.

Audio System Refactoring – Author: Shah Abdul Qadir

I rebuilt the audio system from scratch to squash some major bugs. The worst one was background tracks overlapping when you clicked through menus too fast. You'd end up with three songs playing at once.

Things got even worse when I started packaging the game into a standalone app. The standard JavaFX MediaPlayer uses the OS's native audio decoders. These decoders absolutely refuse to read audio files compressed inside a packaged Java module. So, the exported game had zero background music, even though it worked fine in the IDE. To fix this, I made an AudioManager class and completely dropped MediaPlayer. I switched everything to AudioClip, which loads raw audio data right into RAM and totally bypasses that native OS bug. I also added a check to stop any currently playing song before starting a new one:

```
public void playBackgroundMusic(String path) {  
    try {  
        // Stops the overlap bug by killing the old song first  
        if (backgroundMusic != null) {  
            backgroundMusic.stop();  
        }  
    }  
}
```

```

    }
    URL resourceUrl = getClass().getResource(path);
    if (resourceUrl != null) {
        backgroundMusic = new AudioClip(resourceUrl.toExternalForm());
// Tell the clip to loop forever
        backgroundMusic.setCycleCount(AudioClip.INDEFINITE);
        backgroundMusic.setVolume(0.1);
        backgroundMusic.play();
    }
} catch (Exception e) {
    System.err.println("Error playing BGM: " + e.getMessage());
}
}

```

I built a similar method for sound effects that caches the pop and error sounds so they can fire rapidly during combat without causing lag.

Refactoring for Packaging – Author: Shah Abdul Qadir

A huge chunk of my time went into refactoring the code so we could actually export it. I wanted to turn our raw code into a real .app for Mac and .exe for Windows so no one had to install Java or use a terminal to play it.

To get the Java jlink bundler working, I had to create a module-info.java file in the root folder. This tells the compiler exactly which JavaFX libraries to include so it doesn't pack gigabytes of useless files.

```

module ants.vs.bees
{
// Tell the bundler we need these specific JavaFX libraries
requires javafx.controls;
requires javafx.graphics;
requires javafx.media;
requires javafx.base;
// Open our folders to the compiler
exports core;
exports ants;
}

```

The hardest part by far was fixing the file paths. Standard Java code like `new File("img/start.png")` tries to search your physical hard drive. That crashes the game once it's zipped into an .exe because that hard drive path doesn't exist anymore. I combed through the whole codebase and replaced every image and audio path with a classpath locator using `getResource()`.

```

public static Image loadImage(String filePath) {
    try {
// Add a slash to the front if it is missing
        String safePath = filePath.startsWith("/") ? filePath : "/" + filePath;
// Search inside the packaged zip file for the image
        URL resourceUrl = FileUtils.class.getResource(safePath);
        return new Image(resourceUrl.toExternalForm());
    } catch (Exception e) {
        System.err.println("Image missing: " + e.getMessage());
        return null; // Return null safely instead of crashing the game
    }
}

```

I also had to tweak our Maven pom.xml file so the compiler would actually grab our text files and images during the build. Before I fixed that, Maven just ignored our image folder and the menus loaded totally blank.

Continuous Integration (CI/CD) Pipeline

To make sharing the final game easy, I set up an automated CI/CD pipeline using GitHub Actions. This meant no one had to manually build the game on their own laptops anymore.

I wrote custom YAML scripts that trigger every time we push code to the main branch. GitHub reads the script, fires up a Mac server and a Windows server in the cloud, compiles the code, and builds the installers automatically

```
- name
: Package Windows EXE run
: |
jpackage --type exe --name AntsVsBees --runtime
-image target
/image \
--module ants
.vs.bees
/core.AntsVsSomeBees \
--win
-shortcut --win
-dir
-chooser --win
-per
-user
-install
```

One big roadblock was Windows User Account Control. Windows actively blocks the .exe file with a scary warning screen because we are an "unknown publisher" trying to install in system files. To get around this, I added the --win-per-user-install flag to the script. This forces the game to install into the user's local AppData folder instead, completely bypassing the need for an Admin password.

I also had to clean up our Git history. Early on, compiling these apps created massive 100MB runtime folders that violated GitHub's file size limits and broke the repo. I fixed the .gitignore files and purged the old commits so only the raw code gets pushed to the cloud now

Game System

Game Menus and Pane handling - Author: Parastoo Shojaei Sarcheshmeh

[5] The menu system controls the full lifecycle of the application, from the first screen the player sees to the game ending and results being saved. It is built around a central controller class, GameAppController, which manages screen transitions and connects all the individual menu panes together. Each pane is a self-contained JavaFX layout class responsible only for its own visual structure, while GameAppController handles all the logic and navigation between them.

GameAppController - Author: Parastoo Shojaei Sarcheshmeh

GameAppController is the hub of the menu system. It holds references to the Stage, the shared Scene, and all active pane instances. Reusing a single Scene and swapping its root node avoids recreating the window on every transition:

```

    if (scene == null) {
        scene = new Scene(mainMenuPane, 1024, 768);
        stage.setScene(scene);
        stage.show();
    } else {
        scene.setRoot(mainMenuPane);
    }
}

```

It also stores the currently selected username, difficulty level, and seed so that they are available when the game ends and results need to be saved:

```

private int level = 1;
private String selectedUserName = null;
private long currentSeed = SeedGenerator.seedGenerator();

```

MainMenuPane - Author: Parastoo Shojaei Sarcheshmeh

MainMenuPane is the first screen the player sees. It extends VBox and lays out all controls vertically. It contains a title label, a Start Game button, a user selector dropdown, a new user text field and button, a difficulty dropdown, a seed field, a Settings button, a Score Screen button, and an Exit button.

The background image is applied using a BackgroundImage object loaded from img/menu_bg.png. The difficulty dropdown was renamed from Level to Difficulty to better communicate the gameplay effect to the player:

```

levelSelectDropdown = new ComboBox<>();
levelSelectDropdown.getItems().addAll("Difficulty 1", "Difficulty 2", "Difficulty
3");
levelSelectDropdown.setPromptText("Select Difficulty");

```

A userErrorLabel was added to show a red error message when a player tries to create a username that already exists:

```

userErrorLabel = new Label("");
userErrorLabel.setStyle("-fx-text-fill: red; -fx-font-size: 13px;");

```

The seed field is pre-filled with a value from SeedGenerator each time the menu loads, so every session starts with a unique seed by default. Players can overwrite it to replay a specific game configuration.

All buttons in MainMenuPane are exposed via public getter methods so that GameController can attach actions to them without the pane needing to know what those actions do. This keeps the pane purely responsible for layout.

User Account System - Author: Parastoo Shojaei Sarcheshmeh

When the player creates a new user, GameController first checks whether the name already exists before calling UserManager.createUser():

```

mainMenuPane.getNewUserButton().setOnAction(e -> {
    String newName = mainMenuPane.getNewUserField().getText().trim();
    if (!newName.isEmpty()) {
        if (UserManager.getUser(newName) != null) {
            mainMenuPane.getUserErrorLabel().setText("Username already exists.");
        } else {

```

```

        mainMenuPane.getUserErrorLabel().setText("");
        UserManager.createUser(newName);
        mainMenuPane.getSelectUserDropdown().getItems().clear();
        for (UserAccount u : UserManager.getAllUsers()) {
            mainMenuPane.getSelectUserDropdown().getItems().add(u.getName());
        }
        mainMenuPane.getSelectUserDropdown().setValue(newName);
        mainMenuPane.getNewUserField().clear();
    }
}
});

```

UserManager saves all accounts to users.json using a hand-written JSON serialiser. Each entry stores the player's name, difficulty level, score, passed status, time taken, and seed. This allows the score screen to display full history across sessions.

SeedGenerator - Author: Parastoo Shojaei Sarcheshmeh

SeedGenerator produces a 12-digit numeric seed from the current date and time using the format yyMMddHHmmss. This means two players starting a game at the same second will get the same map layout and bee waves, enabling them to share and compare runs:

```

public static long seedGenerator() {
    DateTimeFormatter formatter = DateTimeFormatter.ofPattern("yyMMddHHmmss");
    long dateTimeInt = Long.parseLong(LocalDateDateTime.now().format(formatter));
    return dateTimeInt;
}

```

HelpScreenPane - Author: Parastoo Shojaei Sarcheshmeh

HelpScreenPane is shown immediately after the player clicks Start Game. It displays a full-screen guide image loaded from img/help_screen.png. A green PLAY button sits at the bottom of the screen. The game does not start until the player clicks it, giving them time to read the guide:

```

Button playButton = new Button("PLAY");
playButton.setFont(Font.font("SansSerif", FontWeight.BOLD, 24));
playButton.setPrefSize(160, 55);
playButton.setStyle(
    "-fx-background-color: #4CAF50;" +
    "-fx-text-fill: white;" +
    "-fx-background-radius: 8;" +
    "-fx-cursor: hand;");
playButton.setOnAction(e -> onFinish.run());

```

When the player clicks PLAY, GameController switches the scene root to the game screen and calls `antGame.startGameLoop()`.

GameScreenPane - Author: Parastoo Shojaei Sarcheshmeh

GameScreenPane wraps the AntGame canvas inside a BorderPane and adds a Pause button in the top-right corner. The button's size scales with the window using property bindings on the stage:

```

pauseButton.prefWidthProperty().bind(stage.widthProperty().multiply(120
/ AntGame.SCENE_WIDTH));

```

```
pauseButton.prefHeightProperty().bind(stage.widthProperty().multiply(20 /
AntGame.SCENE_HEIGHT));
```

PauseMenuPane - Author: Parastoo Shojaei Sarcheshmeh

PauseMenuPane appears as a semi-transparent overlay on top of the game screen when the player presses the Pause button or the Escape key. It is added to the StackPane that wraps the game, so it sits on top without replacing the game view:

```
scene.setOnKeyPressed(e -> {
    if (e.getCode() == KeyCode.ESCAPE) {
        showPauseMenu(gameRoot);
    }
});
```

It contains four buttons: Resume, Reset Game, Return to Main Menu, and Settings. Resume removes the overlay and calls `antGame.resumeGame()`. Return to Main Menu stops the game, saves the session as a loss, and navigates back:

```
pauseMenuPane.getReturnToMainMenuButton().setOnAction(e -> {
    if (antGame != null) {
        antGame.stopGame();
    }
    if (selectedUserName != null) {
        UserManager.updateTime(selectedUserName, GameTimer.getElapsedSeconds());
        UserManager.updateSeed(selectedUserName, currentSeed);
        UserManager.updateLevel(selectedUserName, level);
        UserManager.updateScore(selectedUserName, ScoreManager.getScore());
        UserManager.setPassed(selectedUserName, false);
    }
    showMainMenu();
});
```

SettingsPane - Author: Parastoo Shojaei Sarcheshmeh

SettingsPane is accessible from both the main menu and the pause menu. It contains a music toggle button and a game speed selector. The toggle reads the current music state from `AntGame.isMusicEnabled()` when it opens, so it always reflects the actual state regardless of where it was opened from:

```
boolean currentlyOn = AntGame.isMusicEnabled();
ToggleButton musicToggle = new ToggleButton(currentlyOn ? "Music: ON" : "Music:
OFF");
musicToggle.setSelected(currentlyOn);
```

The speed selector offers two options: Speed 1 at 2 seconds per turn and Speed 2 at 1 second per turn. The active selection is highlighted in green or blue:

```
speed1Button.setOnAction(e -> {
    AntGame.setTurnSeconds(2);
    updateSpeedStyles.run();
});
speed2Button.setOnAction(e -> {
    AntGame.setTurnSeconds(1);
    updateSpeedStyles.run();
});
```


GameOverPane - Author: Parastoo Shojaei Sarcheshmeh

GameOverPane is shown when the game ends, whether the player wins or loses. It displays a full-screen background image — win_bg.png for a win and gameover_bg.png for a loss — with large text overlaid on top. The image is displayed using an ImageView bound to the pane's width and height so it scales correctly with the window. After 4 seconds it automatically navigates to the score screen:

```
javafx.scene.image.ImageView bgView = new javafx.scene.image.ImageView(new
Image(imagePath));
bgView.setPreserveRatio(false);
bgView.fitWidthProperty().bind(this.widthProperty());
bgView.fitHeightProperty().bind(this.heightProperty());
```

Before showing GameOverPane, GameAppController saves the final score and pass status to the user account:

```
antGame.setOnGameOver(playerWon -> {
    if (selectedUserName != null) {
        UserManager.updateScore(selectedUserName, ScoreManager.getScore());
        UserManager.setPassed(selectedUserName, playerWon);
    }
    GameOverPane gameOverPane = new GameOverPane(playerWon, () -> {
        ScoreScreenPane scoreScreen = new ScoreScreenPane(() -> showMainMenu());
        scene.setRoot(scoreScreen);
    });
    scene.setRoot(gameOverPane);
});
```

ScoreScreenPane - Author: Parastoo Shojaei Sarcheshmeh

ScoreScreenPane displays three leaderboard columns, one for each difficulty level. Each column shows the top 10 players sorted by pass status first, then score descending, then time ascending. A Your Last Game section at the bottom shows the most recent entry in users.json. The Back to Menu button returns the player to MainMenuPane.

Score and Timer Systems - Author: Parastoo Shojaei Sarcheshmeh

ScoreManager is a static utility class that tracks the current score. It assigns 10 points for a standard bee, 20 for a ghost bee, and 30 for a zombie bee, all multiplied by the current difficulty level. GameTimer is a matching static class that tracks elapsed time in seconds. Both are reset at the start of each game and read when the session ends to save results.

Graphics – Author: Shah Abdul Qadir

Audio – Author: Shah Abdul Qadir

Difficulty System – Author: Paddy Costelloe

The idea behind the difficulty system was to follow in the footsteps of games like [4] “Noita” that use a seed to randomise their game levels. This seed means that the game level constructs itself the same way for each player entering the game using the same seed. This meant that players could share seeds between themselves and compete to complete various especially hard seed levels.

In the AntvsBees game this seed was used to set random number generators for spawning the hive and how much water spawned in the level. It was also used at one point to vary the bee armour values but during testing this was found to be too extreme and unpredictable if given too much room to change. Too little room to change was not noticeable. So, it was only applied to the enemies’ spawns and the water spawns to give the game variety.

At first the randomiser was created in text form for testing to ensure that the Hive was being built correctly and that the seed worked to reform the spawned hive the same with the same seed given.:

```
public static void randomizeLevel(int level, long seed) {
    //if no seed is provided, use the current date and time to generate a seed for
    randomness.
    // This ensures that each playthrough can be unique while still allowing for
    reproducibility if a seed is given.
    // seed should be 12 digits long to allow for enough variability.
    if(seed == 0) {
        seed = seedGenerator();
    }
    Random rand = new Random(seed);
    int numBees = level * 30;
    int beeArmor = 2 * level;
    System.out.println("Number of bees for level " + level + ": " + numBees);
    System.out.println("Bee armor for level " + level + ": " + beeArmor);
    int numbGhostBees = rand.nextInt((numBees / 3) - (2 * level)) + (2 * level);
    System.out.println("Number of Ghost Bees for level " + level + ": " +
numbGhostBees);
    int numbZombieBees = rand.nextInt((numBees / 3) - (2 * level)) + (2 * level);
    System.out.println("Number of Zombie Bees for level " + level + ": " +
numbZombieBees);
    int numbStandardBees = numBees - numbGhostBees - numbZombieBees;
    System.out.println("Number of Standard Bees for level " + level + ": " +
numbStandardBees);
}
public static long seedGenerator(){
    DateTimeFormatter formatter = DateTimeFormatter.ofPattern("yyyyMMddHHmm");
    long dateTimeInt = Long.parseLong(LocalDate.now().format(formatter));
    return dateTimeInt;
}
```

Once this test method was proven to work and create hives with reliable predictability based on the seed given it was built directly into the Hive class. Then a seed text area was inserted into the menu. If no seed is selected then the seed used is gotten from the time and date. However if a seed is used, it would mean that the player was playing an already generated level.

The actual difficulty comes from the biased spawn of normal bees to special bees. It also built the hive so that normal bees come out first, giving the player time to prepare for the special bees which come in later waves. The hive for level 1 - 20 bees, level 2 - 40 bees and level 3 – 60 bees. Each level also increased the likelihood of water spawning in tiles based on increasing chance level to level.

Game Flow – Author: Paddy Costelloe

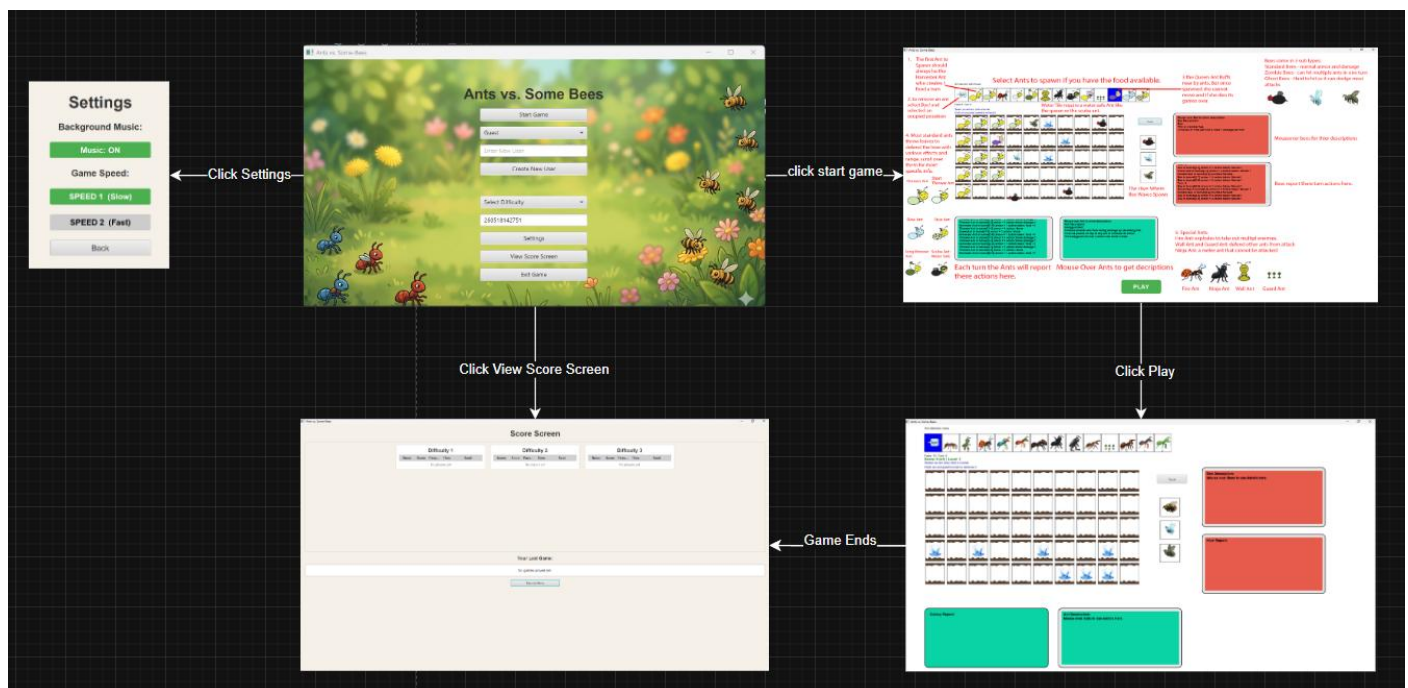


Figure 6 - game scene flow chart

When the game starts the user is met with the main menu, this contains the following buttons:

- Start Game – start a new game with current settings
- Select user – defaults to guest
- Enter New User – text field for adding a new user name
- Create New User – captures the username string from the above text field
- Select Difficulty – defaults to level 1, can select 1,2 or 3 levels of difficulty
- Seed – text field, a user can enter a custom seed to play the same game as another user.
- Settings – turn on and off music and change game speed
- View score screen – takes the user to the saved scores from previous games.
- Exit Game – exits the game

Once a **start game** is selected the menu captures the user and the game level, it then uses the seed random or given to generate the level by building the hive and distributing the water tiles.

A help screen is displayed before the game starts. This is a stand in for proper art developed by a games artist/UI designer outside of this projects remit. The player must select **play** to start the game.

The game is split into turns lasting 1 to 2 seconds depending on game speed. The game speed dictates how fast ant action happen and when bee animations take place. The player can select any ant from the top row. As they hover over the Ant and Bee icons, a mouse over tool tip will pop up and the Ant and Bee Helper terminals will update with pertinent information.

Once an ant is selected, the player can click any unoccupied space to deploy the ant if they have sufficient food and the Ant's requirements are met (e.g. not a water tile if they can't swim) user feedback pops up after each user action underneath the selection bar.

If a queen is deployed, there can be only one.

The game can be paused and reset at any time by selecting the pause button.

The game is lost if a bee reaches the rear of the hive or reaches the queen bee.

The game is won if all the bees in the hive have been exhausted.

Either way the game will progress to the score screen which has a back button that takes the player back to the main menu.

Testing

White Box Testing – junit testing - Author: Parastoo Shojaei Sarcheshmeh

The team adopted a white box testing methodology throughout development, writing JUnit 5 unit tests alongside the implementation of each class. Rather than testing from the outside purely through inputs and outputs, white box testing allowed us to verify internal logic paths, state transitions, and edge cases based on direct knowledge of the code. Every time a new class was created or a method was added, a corresponding JUnit test was written to verify its behaviour before it was merged into the main branch [\[9\]](#).

Methodology - Author: Parastoo Shojaei Sarcheshmeh

Our testing approach followed a consistent structure across all test classes. Each test was written in three stages: arrange, act, and assert. The arrange stage sets up the objects needed for the test. The act stage calls the method being tested. The assert stage verifies the result using

JUnit's assertEquals, assertNull, assertNotNull, assertTrue, assertFalse, or assertDoesNotThrow assertions.

The @BeforeEach annotation was used in tests where multiple test methods needed the same starting state, avoiding repetition and keeping each test independent:

java

```
@BeforeEach
public void setUp() {
    hungryAnt = new HungryAnt();
    testPlace = new Place("testPlace");
    testPlace.addInsect(hungryAnt);
}
```

Rather than writing one large test per class, each test method targets a single behaviour. This makes failures easier to diagnose because the test name immediately identifies what broke.

While AI tools were available during development, the team chose to write all JUnit tests manually. This decision was made to ensure each test reflected a genuine understanding of the class being tested rather than generated boilerplate that might miss edge cases specific to our implementation.

Test Classes - Author: Parastoo Shojaei Sarcheshmeh

AntColonyTest verifies the food system of the colony. testInitialFoodValue checks that the colony stores its starting food correctly. testIncreaseFood checks that calling increaseFood(2) on a colony starting with 5 food results in 7:

```
@Test
void testIncreaseFood() {

    AntColony colony = new AntColony(1, 1, 0, 5, 10, 1);
    colony.increaseFood(2);
}
```

```

    assertEquals(7, colony.getFood(),
        "AntColony should correctly increase food");
}

```

PlaceTest verifies the basic addInsect and removeInsect behaviour of Place. After adding an ant, getAnt() must return that ant. After removing it, getAnt() must return null.

BeeTest tests the two core bee behaviours. testBeeStingsWhenBlocked places a bee and an ant in the same tile and calls bee.action(null), then checks that the ant lost 1 armor. testBeeMovesWhenNotBlocked places a bee in a two-tile tunnel with no ant and verifies it moves to the next place after one action.

HarvesterAntTest verifies that calling action() on a HarvesterAnt increases the colony's food by exactly 1.

ThrowerAntTest tests two cases. testThrowerAntNoTarget verifies no exception is thrown when there is no bee to target. testThrowerAntAttacksBee places a bee in the same tile and verifies it loses 1 armor after one action:

```

@Test
void testThrowerAntAttacksBee() {
    Place place = new Place("testPlace");
    ThrowerAnt ant = new ThrowerAnt();
    Bee bee = new Bee(3);
    place.addInsect(ant);
    place.addInsect(bee);
    ant.action(null);
    assertEquals(2, bee.getArmor(),
        "ThrowerAnt should reduce bee armor by 1");
}
}

```

ShortThrowerAntTest and **LongThrowerAntTest** both set up a multi-place tunnel and verify that getTarget() returns the correct bee or null depending on distance. ShortThrowerAnt must not target bees beyond 2 places. LongThrowerAnt must not target bees closer than 4 places.

SlowThrowerAntTest verifies the slow effect in two steps. First it checks that the bee stays in place on its first slowed turn. Then it checks that the bee moves on the next turn, confirming the every-other-turn behaviour:

java

```

@Test
public void testBeeActsEveryOtherTurnWhenSlowed() {
    slowThrowerAnt.action(null);

    bee.action(null);    assertEquals(beePlace, bee.getPlace(), "Bee should
remain in the same place on the skipped turn");

    bee.action(null);
    assertEquals(antPlace, bee.getPlace(), "Bee should move forward on the next
allowed turn");
}

```

StunThrowerAntTest verifies that a stunned bee skips exactly one turn and then resumes normal movement.

WallAntTest checks that WallAnt initialises with 4 armor and a food cost of 4, and that calling action() does not change the colony's food.

HungryAntTest covers three behaviours. testEatBee checks that after eating, digestTimeLeft is 3 and the bee is removed from the place. testDigestTimeReduction checks that each action call while digesting decrements the counter by 1. testDoesNotEatWhileDigesting verifies that a second bee is not eaten while the ant is still digesting:

Pre-Refactor code:

```
@Test
public void testDoesNotEatWhileDigesting() {
    Bee firstBee = new Bee(3);
    Bee secondBee = new Bee(3);

    testPlace.addInsect(firstBee);
    hungryAnt.action(null);

    testPlace.addInsect(secondBee);
    hungryAnt.action(null);

    assertEquals(2, hungryAnt.getDigestTimeLeft(),
        "Digest time left should decrease to 2 while digesting");
    assertEquals(1, testPlace.getBees().length,
        "Second bee should still be in the place while HungryAnt is digesting");
}
```

BodyguardAntTest uses @BeforeEach to set up a tile containing a BodyguardAnt protecting a ThrowerAnt with a bee present. Three tests verify: the bodyguard is the visible ant, the bodyguard takes damage first, and after the bodyguard dies the thrower ant remains in the place.

FireAntTest gives the FireAnt two bees with 4 armor each, then reduces the ant's armor to zero. It verifies both bees lost exactly 3 armor from the explosion and the ant is dead.

NinjaAntTest places a NinjaAnt with two bees and calls action(). It checks both bees lost 1 armor and that blocksPath() returns false.

GhostBeeTest applies 1000 hits to a GhostBee with very high armor and counts how many actually connected. It then asserts the hit count falls between 400 and 600, statistically confirming the 50% evasion rate:

```
assertTrue(damageTaken > 400 && damageTaken < 600,
    "Ghost Bee should evade roughly 50% of attacks. Hits taken: " +
    damageTaken);
```

ZombieBeeTest sets up two connected tiles with an ant in each and verifies that after one ZombieBee action, both ants lost 1 armor and the zombie did not move.

QueenAntTest verifies the Singleton pattern and the buff mechanic. It confirms the ant is unbuffed before the queen acts, then checks it has doubled damage and is marked as buffed after queenAnt.action(colony).

ScubaThrowerAntTest sets a tile to water and verifies that a standard ThrowerAnt is rejected while a ScubaThrowerAnt deploys successfully.

FileUtilsTest uses @BeforeAll with Platform.startup() to initialise the JavaFX toolkit in the headless test environment, then verifies that images, audio clips, and media players load without returning null.

Black Box Testing – document – Author: Paddy Costelloe

Below are examples of user stories from 3 areas; gameplay, menus and audio. Each user story also comes with a test for acceptance criteria that is used for the testing in black box testing. The testing criteria doubles as the acceptance criteria so that if the test passes it can also be taken as acceptance of a completed task. Some of the areas only have short tests as they inherit from parent classes that are also being tests. Below is a scuba ant user story, it does not check or test to see if the ant can throw a leaf because the scuba ant inherits to throwing of leaves from the thrower ant class. Instead, its specific test area is the deployment to a water tile and the fact that other ants cannot do the same.

For the full list of user stories from this project please see Appendix [\[i\]](#)

Area: Gameplay – Ants Example

Author: Paddy Costelloe Coder: Paddy Costelloe

Short: As a Player, I want an ant that can be deployed to normally unusable water tiles.

User	Action	Expected Outcome or Reason		
Player	I want an ant that can be deployed to normally unusable water tiles. Normal bees cannot go on these tiles as they cannot swim.	A water tile has been added to the game so that I cannot normally deploy ants to it. But I would like to have the option to deploy an ant at a higher cost.		
Testing – Scuba Ant				
Action		Expected result	Pass/Fail	Failure Description
Deploy a Scuba Ant to a water tile.		The ant should deploy as normal if there is enough food and act as a thrower ant, throwing leaves at bees within 3 tiles.	Yes	
Deploy a non-scuba ant to a water tile		You should not be able to deploy a non-scuba ant to a water tile, except for the queen. All other ants cannot swim. You should get a warning message.	Yes	

Similarly to the above example the ghost bee's main ability is to dodge various attacks based on random chance. It does not test any normal Bee functions outside of its area that it has inherited from the bee class. This means the acceptance criteria is specific and testable to each new class as the team builds on the old framework.

Area: Gameplay – Bees Example

Author: Paddy Costelloe Coder: Shah Abdul Qadir

Short: As a Player, I want a new enemy to fight that is harder to hit and so hard to take down before it damages ants.

User	Action	Expected Outcome or Reason		
Player	I want a new enemy to fight that is harder to hit and so hard to take down before it damages ants.	This bee has a chance to dodge attacks meaning when it is incoming you need to put multiple defenders in front of it to take it down.		
Testing – Ghost Bee				
Action		Expected result	Pass/Fail	Failure Description
The ghost Bee should advance and do damage like a normal bee; however, it should dodge the majority of attacks requiring a lot of effort to kill by sheer numbers		Reports dodging 50% of attacks determined randomly.	pass	Can only be visually test and so must be tested multiple times.

Author: Parastoo Shokaei Coder: Parastoo Shokaei

Short: As a player, I want theme music in the game so that the game feels more immersive and enjoyable.

User	Action	Expected Outcome or Reason		
Player	I want theme music in the game so that the game feels more immersive and enjoyable.	Music should play during the game and the player can turn it on and off as they wish from the menu.		
Testing – mouse over events				
Action		Expected result	Pass/Fail	Failure Description
Click the start new game button		Music should immediately start playing	pass	
Click the reset game button		Music should reset with the game.	pass	

Click the setting and then turn off music mid game	The music should stop cleanly and restart again if the button is clicked a second time.		
--	---	--	--

Author: Parastoo Shokaei Coder: Parastoo Shokaei

Short: As a Player I want a way I can track my score for various levels.

User	Action	Expected Outcome or Reason
Player	I want a way I can track my score for various levels.	The player will be shown a score screen at the end of each session as well as be able to access it from the main menu. It will show games scores recorded between each register account.

Testing – mouse over events

Action	Expected result	Pass/Fail	Failure Description
Click the score screen button from the main menu	Takes you to the score screen showing 3 lists for the 3 levels of scores in ascending order.	pass	
The game ends	Takes you to the score screen showing 3 lists for the 3 levels of scores in ascending order. Whether the player won or lost they are shown a score screen where their achievement is show along with the top ten positions.	pass	

Bugs & weaknesses

1. Due to an early decision based on the original code we were given and a group discussion about our preferred coding styles and experience, it was decided to stick with coding the UI and layout directly in the classes and methods. This prevented us from early on moving over an FXML document as most of the team were not familiar with scene builder. This means that the final software version handed over to the client was not as easily maintainable as it might have been if we had moved to FXML early.
2. Naming conventions for the GitHub branches could have followed a better set of rules as later in the project the branches being maintained had a wide variety of names that didn't work well for the team when trying to work with each other's branches.
3. The Game speed and animations do not sync well when changed mid game and we were unable to fix this at the end of the project. This is why the game speed is in the setting instead of on the main screen. When used mid game the bees often animate at different rates and become shaky.
4. We probably could have used more junit testing for more edge cases but due to time constraints we just ensured our junit testing worked in the most basic sense. E.g. classes instantiate correctly and don't throw nulls.

AI Usage Declaration

Paddy Costelloe – I used Ai during team meetings, for debugging code issues and for java documentation breakdown. The meetings took place online over teams and were transcribed. Then the transcription was given to copilot to create meeting notes and actions. The meeting notes were used to help the team track actions and give quick notes on what was discussed. Jira was used to track the tasks.

For coding AI usage was primarily restricted to debugging help and understanding of java documentation. Debugging terminal feedback was given to AI to help track down the source of an issue and understand what was happening. For documentation AI was used to make the documentation more readable or to narrow down a specific use case needed for the project.

The Team maintained code documentation and each member of the team was encouraged ensure they read and understood the code they were using prior to implementation via the documenting process.

Shah - I used AI as a helper tool throughout this project to speed up my workflow and troubleshoot weird bugs. I treated it like an advanced search engine, always making sure to review and test the code it suggested before actually putting it into our game.

I used AI image generation to create the 2D game assets, like the sprite sheets for the Ghost Bee and Scuba Ant. I had to write very specific prompts to ensure the AI generated anatomically correct bugs with the right number of limbs, rather than just random blurry designs. For instance, the AI kept trying to put leaves on the ant's back, so I had to explicitly prompt it to remove them and just give me a clean ant animation. Once the AI gave me the sheets, I manually sliced them up in an editor and coded the animations into JavaFX myself.

AI was also incredibly helpful for the DevOps side. When I was trying to package the game into an .exe, I got stuck on a lot of obscure Maven and jlink errors that are tough to Google. I pasted the crash logs into the AI to figure out why the compiler was failing. It pointed me in the right direction for building the module-info.java file and fixing the resource paths.

Finally, I used AI to help me write the base YAML scripts for the GitHub Actions pipeline. I adjusted the scripts myself to add the specific flags we needed (like the user-install flag to bypass admin rights), but having the AI

help format the basic server commands saved me a ton of time. Ultimately, I did the actual implementation and testing myself to make sure everything actually worked on our machines before submitting it.

Parastoo Shojaei Sarcheshmeh – Throughout this project I used AI tools as a support resource at various points in development, alongside documentation, Stack Overflow, and my own knowledge. AI was helpful for specific problems and questions, and in every case I reviewed and tested any suggestions before including them in the codebase.

During debugging AI was particularly useful. When the background music was multiplying on game restart, I used AI to help diagnose the issue and it pointed me toward the fix of disposing the MediaPlayer before creating a new one. When the GameOverPane background was not scaling correctly with the window, AI suggested the ImageView binding approach which I then implemented and tested myself. These kinds of specific, well-defined problems were where AI added the most value.

AI also helped at various points when working with JavaFX's large API. When implementing components like the SettingsPane toggle, the leaderboard layout, and the pause menu overlay, I used AI to help identify the right classes and properties to use. When designing utility classes like UserManager and ScoreManager, I used AI to help think through structural questions before writing the code myself.

In all cases the final implementation, the testing, and the decision making were my own. I used AI as one tool among several throughout the project, and I made sure that everything I committed I fully understood and could explain.

References

[1] - Asana (2024). *Story Points: Your Guide to Estimating User Stories in Agile* • Asana. [online] Asana. Available at: <https://asana.com/resources/story-points>.

[2] - GeeksforGeeks (2017). *Java Singleton Design Pattern Practices with Examples*. [online] GeeksforGeeks. Available at: <https://www.geeksforgeeks.org/java/java-singleton-design-pattern-practices-examples/>.

[3] - Schwaber, K. and Sutherland, J. (2020). *The Scrum Guide*. [online] Available at: <https://scrumguides.org/docs/scrumguide/v2020/2020-Scrum-Guide-US.pdf>.

[4] - store.steampowered.com. (n.d.). Noita on Steam. [online] Available at: <https://store.steampowered.com/app/881100/Noita/>.

[5] Oracle.com. (2025). JavaFX: Working with Layouts in JavaFX Release 8 - Contents. [online] Available at: <https://docs.oracle.com/javase/8/javafx/layout-tutorial/index.html>.

[6] Refactoring Guru (2014). Observer. [online] Refactoring.guru. Available at: <https://refactoring.guru/design-patterns/observer>.

[7] Oracle.com. (2025). Part I: Handling Events (Release 8). [online] Available at: <https://docs.oracle.com/javase/8/javafx/events-tutorial/events.htm>.

[8] GeeksforGeeks (2016). Observer Design Pattern. [online] GeeksforGeeks. Available at: <https://www.geeksforgeeks.org/system-design/observer-pattern-set-1-introduction/>.

[9] Junit.org. (2026). Overview :: JUnit User Guide. [online] Available at: <https://docs.junit.org/6.0.3/overview.html>.

Appendix

[i] – User stories

Project: AntsVsBees

Pattern of User Story:

As a [type of user], I want [an action], so that [some reason]

Area: Graphics

Author: Paddy Costelloe Coder: Shah Abdul Qadir

Short: As a player, I want to be able to resize the screen and move it around

User	Action	Expected Outcome or Reason		
player	I want to be able to resize the screen and move it around	The player may move the window around the screen and when doing this scale objects in the scene. These objects need to keep relative size.		
Testing – mouse over events				
Action		Expected result	Pass/Fail	Failure Description
Resize the window after the reaching the main game		Objects and text areas in the scene should resize and maintain relative scale and position.	pass	

Author: Paddy Costelloe Coder: Shah Abdul Qadir

Short: As a player, I want to be able to resize the screen and move it around

User	Action	Expected Outcome or Reason
player	I want to be able to resize the screen and move it around	The player may move the window around the screen and when doing this scale objects in the scene. These objects need to keep relative size.
Testing – mouse over events		

Action	Expected result	Pass/Fail	Failure Description
Resize the window after the reaching the main game	Objects and text areas in the scene should resize and maintain relative scale and position.	pass	

Area: Gameplay – Menu's

Author: Paddy Costelloe Coder: Parastoo Shokaei

Short: As a player, I want to be able to create an account to track my progress and compare to other players scores

User	Action	Expected Outcome or Reason
player	I want to be able to create an account to track my progress and compare to other players scores	The player should be able to save a basic user account that tracks scores across each game level. These scores should be saved and be viewable via the score screen

Testing – mouse over events

Action	Expected result	Pass/Fail	Failure Description
Enter text and click create new user	The drop down box should immediately populate with the new username. if a game is played the user will have a score against their name on the score screen.	pass	
Click the drop down box in the main menu and select a name	The name is a previously saved user. If a new game is played that user should get an updated score.	pass	

Author: Paddy Costelloe Coder: Parastoo Shokaei

Short: As a player, I don't want the game to immediately start as I launch it as I might want to make changes to the level or settings.

User	Action	Expected Outcome or Reason
player	I don't want the game to immediately start as I launch it	The player might want to change settings on the game before it launches. This requires a main menu that sets the game variables before it launches.

Testing – mouse over events

Action	Expected result	Pass/Fail	Failure Description
Launch the game from the exe	The game launches to the main menu	pass	
Click start game with guest account	The main game scene launches at level 1	pass	
Click the settings option	The player is taken to a setting screen for audio and game speed.	pass	
select exit the game	The game closes	pass	
Select game level 3 and start game	The game starts at level 3 with 60 bees and extra water tiles	pass	

Author: Parastoo Shokaei Coder: Parastoo Shokaei

Short: As a player, I want to resume the game after pausing so that I can continue from the same state.

User	Action	Expected Outcome or Reason		
player	I want to resume the game after pausing so that I can continue from the same state.	Allows the player to leave the game mid-session to do another task without quitting.		
Testing – mouse over events				
Action		Expected result	Pass/Fail	Failure Description
Click the pause button during a game session		The gameplay halts and shows a pause menu. Not further actions are reported, and no further animations are played.	pass	
Exit the pause menu		The game resumes post pause from the exact same point as the player left off.	pass	

Author: Parastoo Shokaei Coder: Parastoo Shokaei

Short: As a player, I want the game to become more difficult across levels so that progression feels meaningful.

User	Action	Expected Outcome or Reason
Player	I want the game to become more difficult across levels so that progression feels meaningful.	There should be some progression on the game to allow more advanced user a more difficult experience. It also challenges the users to compete at a higher level.

Testing – mouse over events			
Action	Expected result	Pass/Fail	Failure Description
From the main menu the player selects level 1 and starts a game	The game should spawn a random assortment of 20 bees and less water tiles.	pass	
From the main menu the player selects level 2 and starts a game	The game should spawn a random assortment of 40 bees and an average amount of water tiles.	pass	
From the main menu the player selects level 3 and starts a game	The game should spawn a random assortment of 60 bees and a lot of water tiles.	pass	

Author: Parastoo Shokaei Coder: Parastoo Shokaei

Short: As a Player I want to have an account in the app so I can track my progress against other players

User	Action	Expected Outcome or Reason		
Player	I want to have an account in the app so I can track my progress against other players	A player should be able to easily set up an account in the app for score tracking. There is no security or tracking in this account as it is purely for tracking scores and achievements locally.		
Testing – mouse over events				
Action		Expected result	Pass/Fail	Failure Description
In the main menu, enter a name and click register new user		Your user name should pop up in the drop-down list of users.	pass	
In the main menu after exiting and re-entering the game click the drop-down menu and select my user account again.		You user account should still be present as well as any score recorded on the score screen.	pass	
Select a user and then complete a game on any level		Your user name and score should be present on the score screen even if you did not score top ten.	pass	

Author: Parastoo Shokaei Coder: Parastoo Shokaei

Short: As a Player I want a way I can track my score for various levels.

User	Action	Expected Outcome or Reason		
Player	I want a way I can track my score for various levels.	The player will be shown a score screen at the end of each session as well as be able to access it from the main menu. It will show games scores recorded between each register account.		
Testing – mouse over events				
Action		Expected result	Pass/Fail	Failure Description
Click the score screen button from the main menu		Takes you to the score screen showing 3 lists for the 3 levels of scores in ascending order.	pass	
The game ends		Takes you to the score screen showing 3 lists for the 3 levels of scores in ascending order. Whether the player won or lost they are shown a score screen where there achievement is show along with the top ten positions.	pass	

Author: Parastoo Shokaei Coder: Parastoo Shokaei

Short: As a player, I want a help or guide menu so that I can understand what each ant and bee does.

User	Action	Expected Outcome or Reason		
Player	I want to view a help screen before playing the game			
Testing – mouse over events				
Action		Expected result	Pass/Fail	Failure Description
Click the start new game button in the menu		A help screen should flash up and show the player a quick help screen before the main game starts	pass	
Click the rest button		A help screen should flash up and show the player a quick help screen before the main game starts	pass	
Mouse over a Bee or Ant		A short mouser over tooltip is shown and the Ant and Bee helper terminals should update with details on what each unit does.	pass	

Author: Parastoo Shokaei Coder: Parastoo Shokaei

Short: As a player, I want theme music in the game so that the game feels more immersive and enjoyable.

User	Action	Expected Outcome or Reason		
Player	I want theme music in the game so that the game feels more immersive and enjoyable.	Music should play during the game and the player can turn it on and off as they wish from the menu.		
Testing – mouse over events				
Action		Expected result	Pass/Fail	Failure Description
Click the start new game button		Music should immediately start playing	pass	
Click the reset game button		Music should reset with the game.	pass	
Click the setting and then turn off music mid game		The music should stop cleanly and restart again if the button is clicked a second time.	pass	

Area: Gameplay – user Feedback

Author: Paddy Costelloe Coder: Paddy Costelloe

Short: As a developer, I need to give users quick feedback as the move around the game screen.

User	Action	Expected Outcome or Reason		
developer	I need to give users quick feedback as the move around the game screen	When a new user comes to the game they want to be able to access quick and easy information about the game features. This will be facilitated by mouseover updates so that as they navigate the area, various information UI's on screen will update.		
Testing – mouse over events				
Action		Expected result	Pass/Fail	Failure Description
When I mouse over a bee or an ant, the info panels will update for each group. This panel will give info on what the ants and bees do as well as their armour values.		When mousing over the terminal update	pass	

When I mouse over a bee or an ant, I should get a short tool tip with the ant or bee's name.	When mousing over the tool tip is made visible and when this end, the tool tip is invisible again. it should also update the name of the ant or bee being moused over.	pass	
--	---	------	--

Author: Paddy Costelloe Coder: Paddy Costelloe

Short: As a developer, I need want the player to know what is happening turn to turn in the absence of an art team to create animations.

User	Action	Expected Outcome or Reason		
developer	I need want the player to know what is happening turn to turn in the absence of an art team to create animations.	Without an art team we are reduced to AI animations or text feedback. So a text area is needed to feedback to the user to allow them to know what is happening each turn.		
Testing – Turn terminals – ants and bees				
Action		Expected result	Pass/Fail	Failure Description
Every turn the turn terminals for ants and bees should update with actions		Each action of ants and bees updates a report string and is printed to the turn terminal broken up the turn numbers.	pass	

Author: Paddy Costelloe Coder: Shah Abdul Qadir

Short: As a developer, I need to feedback to the player when they do things right and wrong.

User	Action	Expected Outcome or Reason		
developer	I need to feedback to the player when they do things right and wrong.	When a player does something right or wrong we want to give them some sort of feedback in absence or art and design. This feedback is designed to be replaced if the game is developed professionally as we are missing art and UI design teams.		
Testing – user feedback bar				
Action		Expected result	Pass/Fail	Failure Description
The user tries to deploy an ant to a bad area (water or an already occupied area)		A red feedback prompt pops up to tell the user they cannot perform that action.	pass	

The user tried to deploy an ant without enough food.	A red feedback prompt pops up to tell the user they need more food.	pass	
The user successfully deploys an ant	A green feedback prompt pops up to tell the user they were successful.	pass	

Area: Gameplay – Bees

Author: Paddy Costelloe Coder: Shah Abdul Qadir

Short: As a Player, I want a new enemy to fight that is harder to hit and so hard to take down before it damages ants.

User	Action	Expected Outcome or Reason		
Player	I want a new enemy to fight that is harder to hit and so hard to take down before it damages ants.	This bee has a chance to dodge attacks meaning when it is incoming you need to put multiple defenders in front of it to take it down.		
Testing – Ghost Bee				
Action		Expected result	Pass/Fail	Failure Description
The ghost Bee should advance and do damage like a normal bee; however it should dodge the majority of attacks requiring a lot of effort to kill by sheer numbers		Reports dodging 50% of attacks determined randomly.	pass	

Author: Paddy Costelloe Coder: Shah Abdul Qadir

Short: As a Player, I want a new enemy to fight that can damage more ants to make the game harder.

User	Action	Expected Outcome or Reason		
Player	I want a new enemy to fight that can damage more ants to make the game harder.	This bee can attack multiple ant locations and required a greater defence.		
Testing – Zombie Bee				
Action		Expected result	Pass/Fail	Failure Description

The zombie bee should act like a normal bee but when it is within 1 tile of an ant it will attack the 3 closest tiles.	Every ant within 2 tiles of the zombie bee should take 1 damage each.	pass	
--	---	------	--

Area: Gameplay – Ants

Author: Paddy Costelloe Coder: already in game

Short: As a Player, I want to be able to remove ants to make changes to my deployment

User	Action	Expected Outcome or Reason		
Player	I want to be able to remove ants to make changes to my deployment	In the early game I will deploy cheap ants and a lot of harvesters. But as the game goes on and I stockpile food I may want to change this deployment to more expensive ant types.		
Testing – Guard Ant				
Action		Expected result	Pass/Fail	Failure Description
Click the bye button and then click and occupied ant tile.		The ant is removed and the user is informed of the successful removal.	pass	

Author: Paddy Costelloe Coder: Parastoo Shokaei

Short: As a Player, I want to be able to defend my most valuable ants to give them more armour.

User	Action	Expected Outcome or Reason		
Player	I want to be able to defend my most valuable ants to give them more armour. The guard ant should sit on top of the normal ant and buff it, dying first before the original ant.	The new guard ant sits on top of other ants in the same space buffing the armour value and dying first.		
Testing – Guard Ant				
Action		Expected result	Pass/Fail	Failure Description

Deploy a guard ant to a occupied space.	Even though the space is occupied the guard ant will still deploy, putting its image on top of the other ant. Also it should die first. The other ant should still be able to act.	pass	
---	--	------	--

Author: Paddy Costelloe Coder: Paddy Costelloe

Short: As a Player, I want an ant that can be deployed to normally unusable water tiles.

User	Action	Expected Outcome or Reason
Player	I want an ant that can be deployed to normally unusable water tiles. Normal bees cannot go on these tiles as they cannot swim.	A water tile has been added to the game so that I cannot normally deploy ants to it. But I would like to have the option to deploy an ant at a higher cost.

Testing – Scuba Ant

Action	Expected result	Pass/Fail	Failure Description
Deploy a Scuba Ant to a water tile.	The ant should deploy as normal if there is enough food and act as a thrower ant, throwing leaves at bees within 3 tiles.	pass	
Deploy a non scuba ant to a water tile	You should not be able to deploy a non-scuba ant to a water tile, except for the queen. All other ants cannot swim. You should get a warning message.	pass	

Author: Paddy Costelloe Coder: Parastoo Shokaei

Short: As a Player, I want an ant that can instantly defeat the most power bees but at a high cost.

User	Action	Expected Outcome or Reason
Player	I want an ant that can instantly defeat the most power bees but it needs to come at a high cost not just in food but in strategy.	Most ants throw leaves and I want a more offensive option to deal with powerful bees other than just spamming throwers.

Testing – hungry Ant

Action	Expected result	Pass/Fail	Failure Description
--------	-----------------	-----------	---------------------

Deploy the hungry ant in front of 2 bees in a tunnel	The hungry ant should immediately remove the bee from play. However, it should have to wait a number of turns before it can go again meaning it gets hit by the second bee.	pass	
--	---	------	--

Author: Paddy Costelloe Coder: Shah Abdul Qadir

Short: As a Player, I want a close quarters ant that cannot be hit but is very expensive to deploy.

User	Action	Expected Outcome or Reason	
Player	I want a close quarters ant that cannot be hit but is very expensive to deploy. This is a melee ant that can only hit bees in its area but can dodge in coming attacks	The Ninja takes up a space and can only attack bees on top of it. Its major advantage is it cannot be hit by normal bees.	
Testing – Ninja Ant			
Action	Expected result	Pass/Fail	Failure Description
Deploy ninja ant in front of bee	If food is available, the ninja deploys and the bee continues forward. If the bee tries to pass the ninja it will take 1 damage, but no other actions will occurs and the ninja will not take damage.	pass	

Author: Paddy Costelloe Coder: Paddy Costelloe

Short: As a Player, I want a queen bee to buff my troops make the game more interesting.

User	Action	Expected Outcome or Reason		
Player	I want a queen bee to buff my troops make the game more interesting.	The queen bee will make troops stronger but also need defending as it is the lynch pin the colony.		
Testing – Stun Thrower Ant				
Action		Expected result	Pass/Fail	Failure Description
Deploy short thrower ant with 6 food		The short thrower appears in an empty space to await a bee to detonate it.	pass	
Deploy short thrower ant no food		The short thrower does not deploy, and a warning message is given to the player	pass	

When ever the queen ant takes an action, any ants in adjacent lanes should get double damage.	Ants in adjacent lanes report double damage in the ant reporting area.	pass	
If the queen dies the games is lost	The moment the queen dies the game stop and a score screen is shown with a failure.	pass	

Author: Paddy Costelloe Coder: Parastoo Shokaei

Short: As a Player, I want a thrower ant that stuns bees pinning them in place.

User	Action	Expected Outcome or Reason		
Player	I want a thrower ant that stuns bees pinning them in place.	When I have an economy working well I want a strategic ant I can deploy to pin bees down in place.		
Testing – Stun Thrower Ant				
Action		Expected result	Pass/Fail	Failure Description
Deploy short thrower ant with 4 food		The short thrower appears in an empty space to await a bee to detonate it.	pass	
Deploy short thrower ant no food		The short thrower does not deploy, and a warning message is given to the player	pass	
A Bee gets within 3 places of the Stun thrower ant		A leaf is thrown at the bee once per turn depleting its armour by 1. It also makes the Bee pause for 2 turns.	pass	

Author: Paddy Costelloe Coder: Parastoo Shokaei

Short: As a Player, I want a thrower ant that slows the bees down to give me more time to set up defence.

User	Action	Expected Outcome or Reason		
Player	I want a thrower ant that slows the bees down to give me more time to set up defence.	This ant both attacks and defends by damaging and slowing bees down.		
Testing – Slow Thrower Ant				
Action		Expected result	Pass/Fail	Failure Description
Deploy short thrower ant with 4 food		The short thrower appears in an empty space to await a bee to detonate it.	pass	

Deploy short thrower ant no food	The short thrower does not deploy, and a warning message is given to the player	pass	
A Bee gets within 3 places of the slow thrower ant	A leaf is thrown at the bee once per turn depleting its armour by 1. It also makes the Bee pause for 1 turn.	pass	

Author: Paddy Costelloe Coder: Parastoo Shokaei

Short: As a Player, I want a more powerful thrower ant to deploy in the end game.

User	Action	Expected Outcome or Reason		
Player	I want a more powerful thrower ant to deploy in the end game.	When I have an economy working I want a more powerful thrower ant to deploy to hit targets further away.		
Testing – long Thrower Ant				
Action		Expected result	Pass/Fail	Failure Description
Deploy short thrower ant with 5 food		The short thrower appears in an empty space to await a bee to detonate it.	pass	
Deploy short thrower ant no food		The short thrower does not deploy, and a warning message is given to the player	pass	
A Bee gets within 4 places of the long thrower ant		A leaf is thrown at the bee once per turn depleting its armour by 1.	pass	

Author: Paddy Costelloe Coder: Parastoo Shokaei

Short: As a Player, I want a slightly weaker and cheaper thrower ant to deploy in the early game.

User	Action	Expected Outcome or Reason		
Player	I want a cheaper early game thrower	An ant I can deploy at the beginning of the game that costs less food but protects the colony while I set up production.		
Testing – Short Thrower Ant				
Action		Expected result	Pass/Fail	Failure Description
Deploy short thrower ant with 3 food		The short thrower appears in an empty space to await a bee to detonate it.	pass	

Deploy short thrower ant no food	The short thrower does not deploy, and a warning message is given to the player	pass	
A Bee gets within 2 places of the short thrower ant	A leaf is thrown at the bee once per turn depleting its armour by 1.	pass	

Author: Paddy Costelloe Coder: Shah Abdul Qadir

Short: As a Player, I want to be able to deploy an area of effect ant in an emergency to wipe out a large enemy group.

User	Action	Expected Outcome or Reason		
Player	Deploy a fire ant that detonates when a bee gets nearby. It also takes out any nearby bees in close by squares.	All bees withing range of the explosion get killed and the fire ant is destroyed.		
Testing – Fire Ant				
Action		Expected result	Pass/Fail	Failure Description
Deploy fire ant with 4 food		The fire ant appears in an empty space to await a bee to detonate it.	pass	
Deploy fire ant no food		The fire ant does not deploy, and a warning message is given to the player	pass	
A bee draws near the fire ant		The fire ant detonates with a small explosion effect, killing all bees in the vicinity. The fire ant is removed from play post detonation.	pass	

Author: Paddy Costelloe Coder: Paddy Costelloe

Short: As a Player, I want to be able to deploy a ranged unit to attack the invaders before they can get to my weaker ants

User	Action	Expected Outcome or Reason		
Player	I want to be able to deploy a ranged unit to attack the invaders before they can get to my weaker ants	Most Ants are weak and undefended up close so the thrower ant uses leaves to attack the incoming bees.		
Testing – Thrower Ant				
Action		Expected result	Pass/Fail	Failure Description
Deploy Thrower Ant		If 4 food is available deploy the ant to an empty space.	pass	

Deploy Thrower Ant no food	If less than 4 food is available the thrower ant does not deploy and a warning message is given the player.	pass	
Bee in range of thrower	The thrower ant should throw a leave when a bee is within 3 spaces once per turn.	pass	

Author: Paddy Costelloe Coder: Paddy Costelloe

Short: As a Player, I to be able to gain more food to deploy more ants.

User	Action	Expected Outcome or Reason		
Player	I want to be able to create an ant that creates me more food so I can deploy more powerful ants later in the game	When the harvester ant is deployed it will collect food for the colony but it has no offensive capabilities and so requires defending.		
Testing – Harvester Ant				
Action		Expected result	Pass/Fail	Failure Description
Player		When I click a harvester ant with 2 food and click an empty space it deploys and start collecting 1 food a turn.	pass	
Player		When I click the harvester ant without enough food it will not deploy and give the player a warning message.	pass	

Author: Paddy Costelloe Coder: Parastoo Shokaei

Short: As a Player, I want to be able to create a wall ant that acts as a blocker/defender unit so that I can defend my other units

User	Action	Expected Outcome or Reason		
Player	I want to be able to create a wall ant that acts as a blocker/defender unit to that I can defend my other units because all my other units are weak and die in one or two hits.	When I deploy the wall ant it sits in front of my other ants to defend them from damage, allowing them to attack uninterrupted and saving me food. The wall ant itself should not attack as it main strategic value should come from blocking damage.		
Testing – Wall Ant				
Action		Expected result	Pass/Fail	Failure Description

Click Wall Ant in the selector and click and empty place with enough food to deploy	The wall ant image appears and an bees get stuck on the wall ant until it runs out of amour.	pass	
Click Wall Ant in the selector and click and empty place without food to deploy	The wall ant should not appear, and a red warning should pop up telling you, you don't have enough food.	pass	
Bees reaching the wall ant cannot pass.	The bee should get stuck in place until the wall ants amor is depleted.	pass	

[ii] – Jira Task back log completed, access given Suraj: <https://software-engineering-csym026-group1.atlassian.net/browse/SCRUM-5>